

Umgang mit dem Kursmaterial

- Sämtliches Kursmaterial (Texte, Folien, Abbildungen, Aufgaben, Lösungen usw.) unterliegt dem Urheberrecht des Dozenten und/oder weiterer Rechteinhaber.
- Die Nutzung ist ausschließlich den Studierenden der angegebenen Fachhochschule im Rahmen des angegebenen Kurses und nur zu Ausbildungszwecken gestattet.
- Jede weitergehende Nutzung, insbesondere das Veröffentlichen, Vervielfältigen, Verbreiten, Teilen oder öffentliche Zugänglichmachen des Materials (z.B. auf Webseiten, in Cloud-Speichern, per E-Mail, in Präsentationen oder Publikationen), ist ohne ausdrückliche schriftliche Zustimmung untersagt und kann rechtliche Konsequenzen nach sich ziehen.
- Die Nutzung des Kursmaterials in Verbindung mit Sprachmodellen oder KI-Systemen (z.B. LLMs wie ChatGPT) ist nur dann zulässig, wenn sichergestellt ist, dass die Inhalte nicht zum Training, zur Speicherung oder zu einer sonstigen Weiterverarbeitung durch Dritte verwendet werden (wie z.B. dem LLM-Anbieter) und alle oben genannten Nutzungsbeschränkungen eingehalten werden.



Kapitel_06

Prozesse

Betriebssysteme

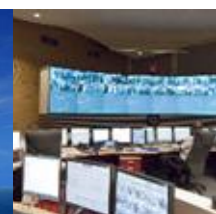
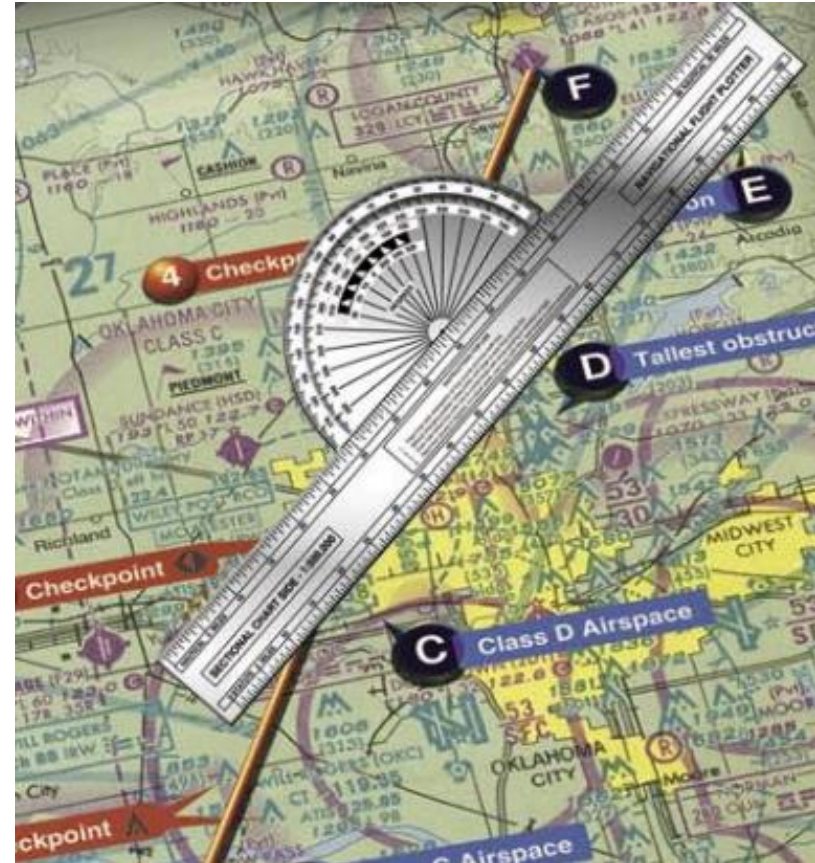
Prof. Dr. Claus Fühner
Fakultät Informatik

Ostfalia Hochschule für angewandte Wissenschaften
Braunschweig/Wolfenbüttel

Prozesse

Prozesse

- Verwaltung
- Zustände
- Dispatching (Umschaltung)
- Prozesse erzeugen, beenden & unterbrechen
- Threads
- Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
 - Strategien
 - Metriken und Vergleich

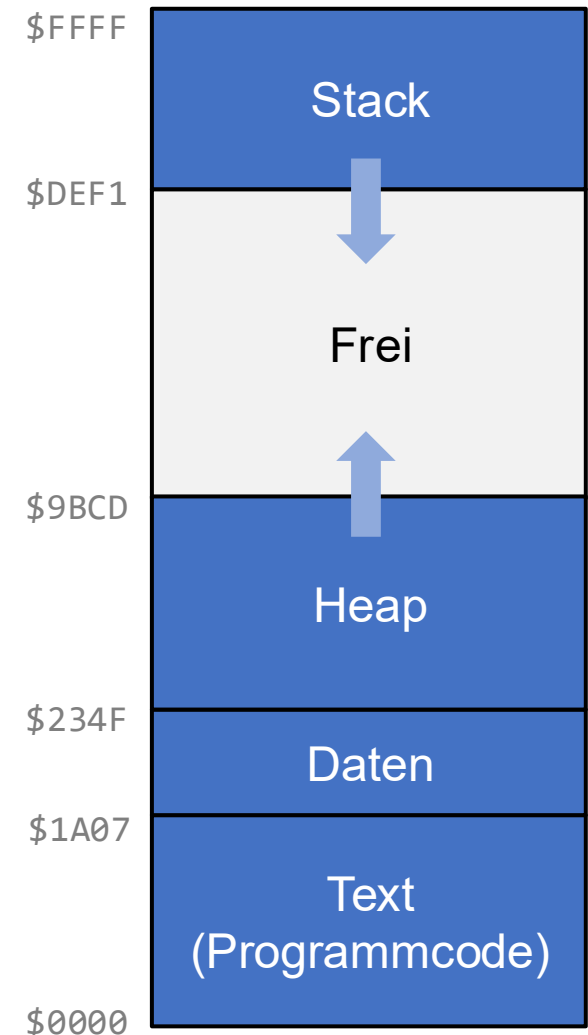


Prozessbegriff

- Ein **Prozess** ist ein Programm (ausführbare Datei) in Ausführung.
- Jedem Prozess wird
 - ein Bereich im **Arbeitsspeicher (Adressraum)** und
 - weitere Ressourcen wie offene Dateien zugeordnet und
 - ein **Teil der verfügbaren CPU-Zeit (→ eigener Kontrollfluss)**

Speicherabbild eines Prozesses (vereinfacht)

1. Textsegment: **Programmcode**
(**Machineninstruktionen**)
2. Datensegment
 - **globale Variablen**
 - statische Belegung – alle Variablen sind mit ihrer Größe zur Compile-Zeit bekannt
3. Heapsegment
 - **dynamische Variablen** – Datenstrukturen, die erst zur Laufzeit aufgebaut werden
 - kann oft selbst dynamisch wachsen
 - oft Teil des Datensegments
4. Stack
 - **Rücksprungadressen, Parameter und lokale Variablen** von Methodenaufrufen
 - kann oft selbst dynamisch wachsen

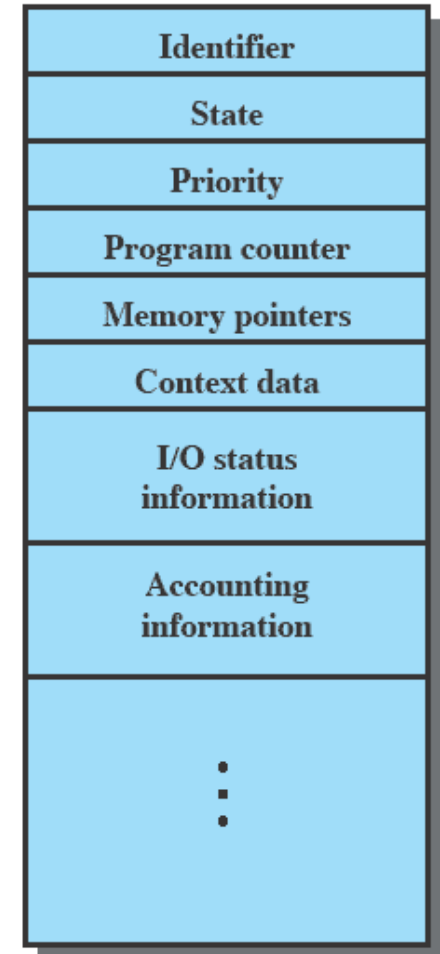


Prozessbegriff

- Ein **Prozess** ist ein Programm (ausführbare Datei) in Ausführung.
- Jedem Prozess wird
 - ein Bereich im **Arbeitsspeicher (Adressraum)** und
 - weitere Ressourcen wie offene Dateien zugeordnet und
 - ein **Teil der verfügbaren CPU-Zeit (→ eigener Kontrollfluss)**
- Interaktive Prozesse (Maus, Tastatur, ...) vs Hintergrundprozesse (**Daemon**)
- Quasiparallelität durch **schnelle Umschaltung** der CPU zwischen Prozessen (Linux: 100-1000 Umschaltungen pro Sekunde)
 - Betriebssystem entscheidet, wann welche Prozesse unterbrochen/fortgesetzt werden
 - Genauer Zeitablauf eines Prozesses nicht vorhersehbar (langsamer immer möglich)
- Zur Verwaltung von Prozessen (und um das Umschalten zu ermöglichen) verwendet das Betriebssystem eine Datenstruktur, die **Prozesskontrollblock (PCB)** genannt wird.
- PCB muss alle Infos enthalten für Unterbrechung und spätere Fortsetzung eines laufenden Prozesses, *ohne das dieser es bemerkt*.

Prozesskontrollblock (PCB)

- **Identifikator** (Prozess-ID, **PID**)
- **Zustand** (bereit, laufend, blockiert, ...)
- **Priorität** (relativ zu anderen Prozessen)
- **Programmzähler**
- **Kontext** (Kopie der CPU-Register, d.h. Statusregister, Universalregister, ...)
- **Speicherabbild** (Zeiger auf Text-, Daten- und Stacksegment)
- **I/O Zustandsinformationen**, u.a.
 - offene I/O-Anfragen,
 - I/O-Geräte, die dem Prozess zugeordnet sind
 - geöffnete Dateien
- **Allgemeine Verwaltungsinformationen**
 - genutzte Ressourcen (z.B. CPU-Zeit)
 - maximal verfügbare Ressourcen



Prozesse unter Linux anzeigen

every (alle Prozesse, auch fremder Nutzer)
full (mehr Spalten, -F oder -o auch nützlich)

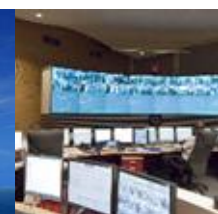
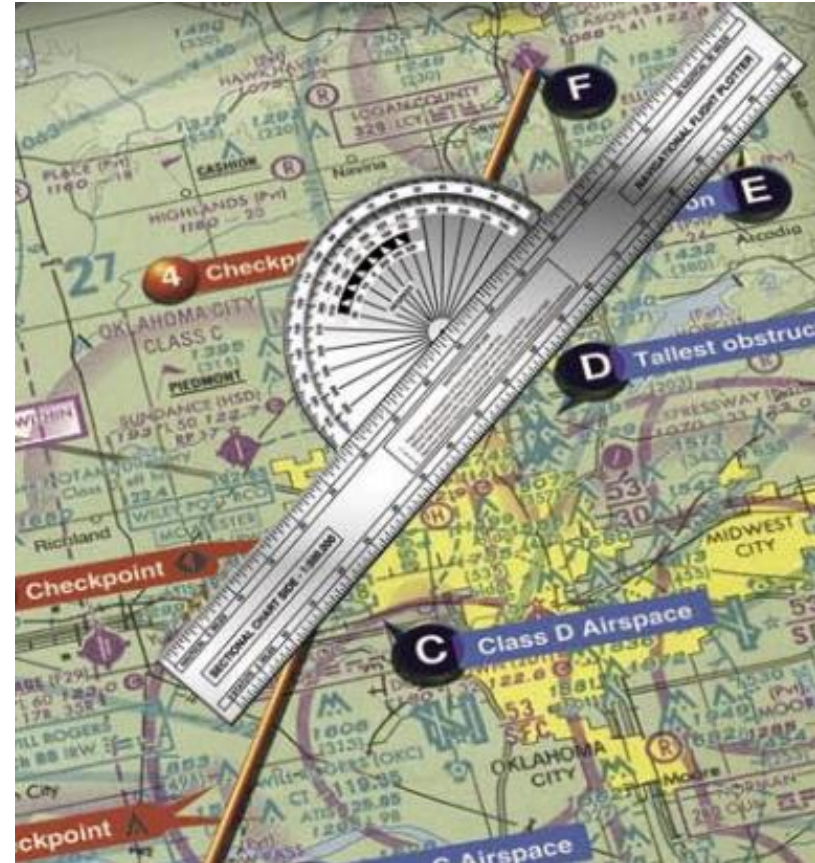
- Prozesse anzeigen unter Linux mit `ps -ef` (oder `-F` oder anderen Optionen)

```
fuehner@bs-ws25:~$ ps -eo user,pid,ppid,stat,tt,time,cmd | grep -v "\["  
USER      PID     PPID  STAT TT          TIME CMD  
root         1         0 Ss   ?           00:00:22 /sbin/init  
root        302         1 Ss   ?           00:00:05 /usr/lib/systemd/systemd-journald  
systemd+    348         1 Ssl  ?           00:00:03 /usr/lib/systemd/systemd-timesyncd  
root        356         1 Ss   ?           00:00:01 /usr/lib/systemd/systemd-udevd  
root        611         1 Ss   ?           00:00:00 /usr/libexec/bluetooth/bluetoothd  
root        612         1 Ss   ?           00:00:01 /usr/sbin/cron -f  
message+    613         1 Ss   ?           00:00:08 /usr/bin/dbus-daemon --system --address=sys  
polkitd     616         1 Ssl  ?           00:00:15 /usr/lib/polkit-1/polkitd --no-debug --log-  
root        619         1 Ss   ?           00:00:04 /usr/lib/systemd/systemd-logind  
avahi       623        610 S    ?           00:00:00 avahi-daemon: chroot helper  
root        659         1 Ssl  ?           00:01:05 /usr/sbin/NetworkManager --no-daemon  
root        661         1 Ss   ?           00:00:02 /usr/sbin/wpa_supplicant -u -s -O DIR=/run/  
root        688         1 Ssl  ?           00:00:00 /usr/sbin/ModemManager  
root        833         1 Ss+  tty1        00:00:00 /sbin/agetty -o -- \u --noreset --noclear -  
fuehner    322600         1 Ss   ?           00:00:00 /usr/lib/systemd/systemd --user  
fuehner    322602    322600 S    ?           00:00:00 (sd-pam)  
fuehner    322620    322600 Ss   ?           00:00:00 /usr/bin/mpris-proxy  
fuehner    322624    322600 Ss   ?           00:00:00 /usr/bin/dbus-daemon --session --address=sy  
fuehner    322626    322595 S    ?           00:00:00 sshd-session: fuehner@pts/0  
fuehner    322627    322626 Ss   pts/0       00:00:00 -bash  
fuehner    322654    322627 R+   pts/0       00:00:00 ps -eo user,pid,ppid,stat,tt,time,cmd  
fuehner@bs-ws25:~$
```

- Wie geht das in Windows? MacOS?

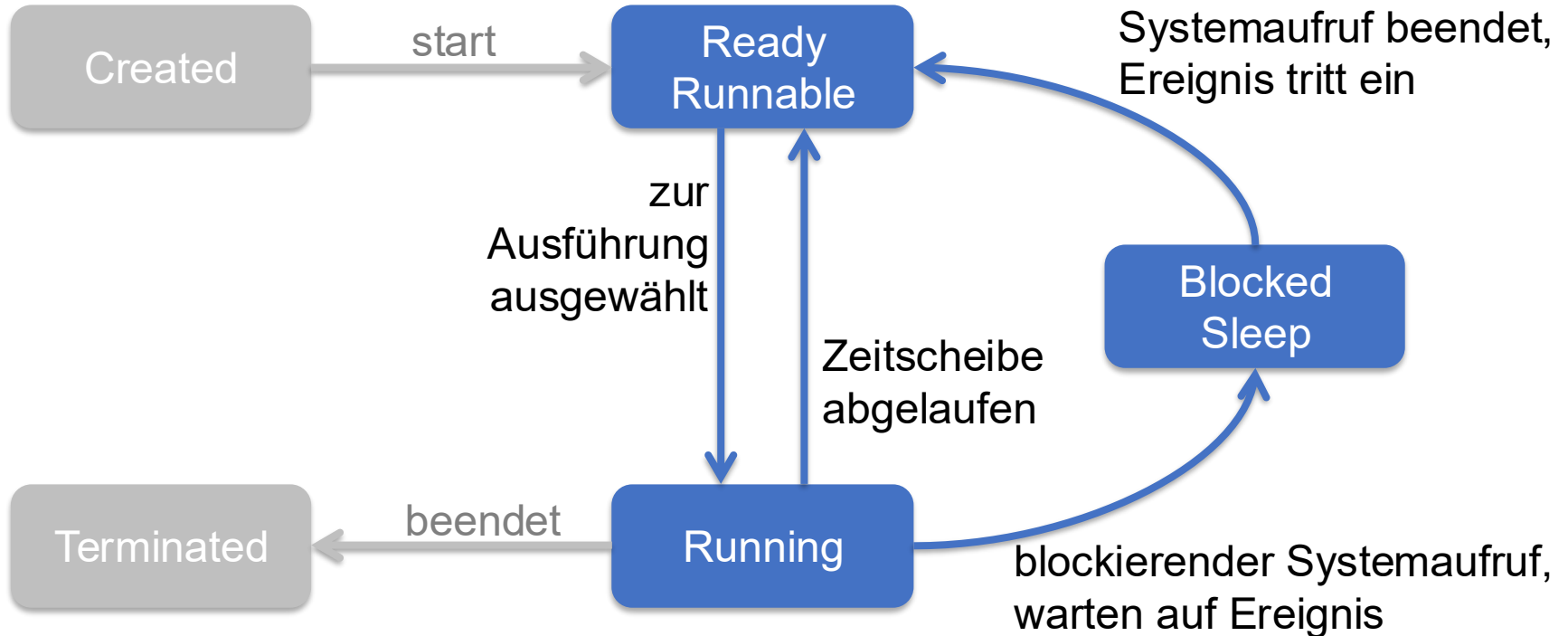
Prozesse

- Prozesse
 - Verwaltung
 - Zustände
 - Dispatching (Umschaltung)
 - Prozesse erzeugen, beenden & unterbrechen
 - Threads
- Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
 - Strategien
 - Metriken und Vergleich





Zustandsdiagramm



- Genaues Zustandsdiagramm abhängig vom Betriebssystem
- JVM hat eigenes Zustandsdiagramm
- `ps` zeigt Zustand in Spalte `stat` an

Der Zustand Blockiert

- Systemaufrufe können einen Prozess in den Zustand **Blocked** bringen
- Fast immer sind **die meisten Prozesse blockiert**:
 - **Warten** auf Ein-/Ausgabe eines Gerätes wie z.B. Tastatur, Netzwerk oder Festplatte
 - **Warten** auf das Ergebnis eines anderen Prozesses
- Systemaufrufe sind entweder **blockierend** oder **nicht blockierend**
- Beispielhafter Ablauf für blockierendes Lesen aus einer Datei mit **read()**:
 1. Prozess **P** ist im Zustand **Running** und ruft **read()** auf
 2. Kernel-Funktion **read** fordert Daten bei Disk-Controller an und kennzeichnet Prozess als **Blocked**
 3. System führt andere Prozesse aus
 4. Disk-Controller erzeugt **Interrupt**, wenn Daten bereit
 5. **Interrupt-Serviceroutine (ISR)** im Kernel weckt **P**, Zustandswechsel nach **Ready**
 6. System kann **P** wieder ausführen



Wie häufig blockieren Prozesse - eher Ready oder Blocked?

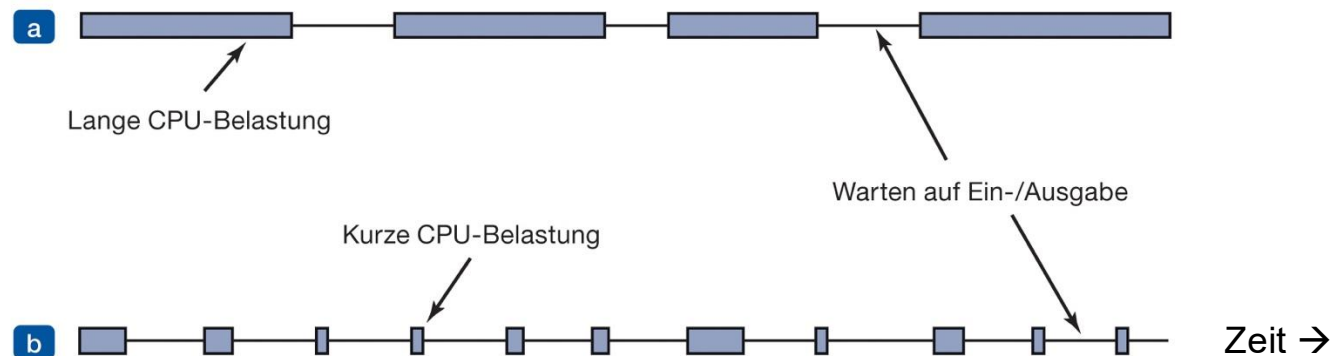
CPU-intensive Prozesse (CPU bound, CPU-lastig)

- lange CPU-Nutzungszeiten und selten Ein-/Ausgabe-Wartezeiten.

I/O-intensive Prozesse (I/O bound, E/A-intensiv oder -lastig)

- häufige E-/A-Wartezeiten (z.B. durch die Festplatte)
- kurze CPU-Nutzungszeiten

Beispiel

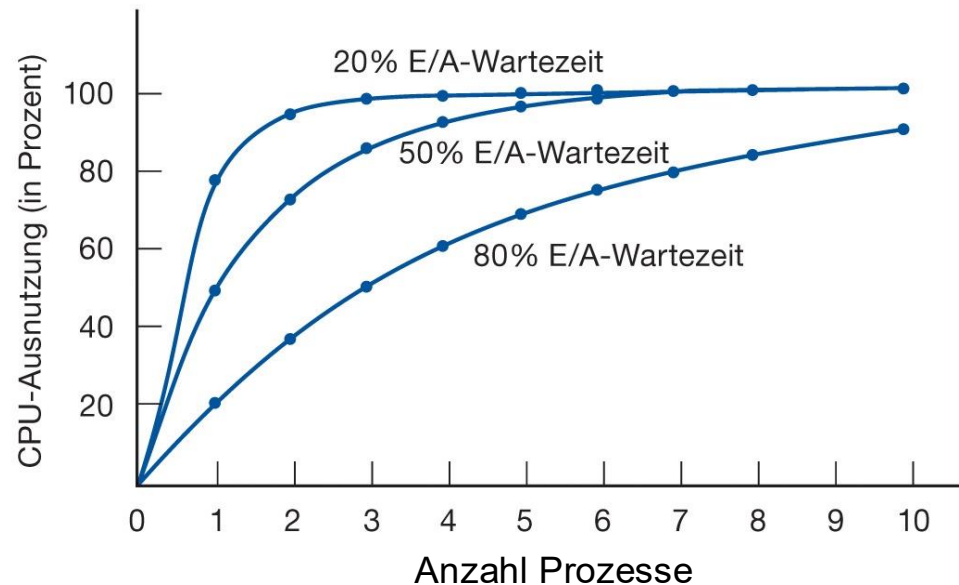


- Durch immer schnellere CPUs werden Prozesse immer E/A-intensiver.



Wie häufig blockieren Prozesse - eher Ready oder Blocked?

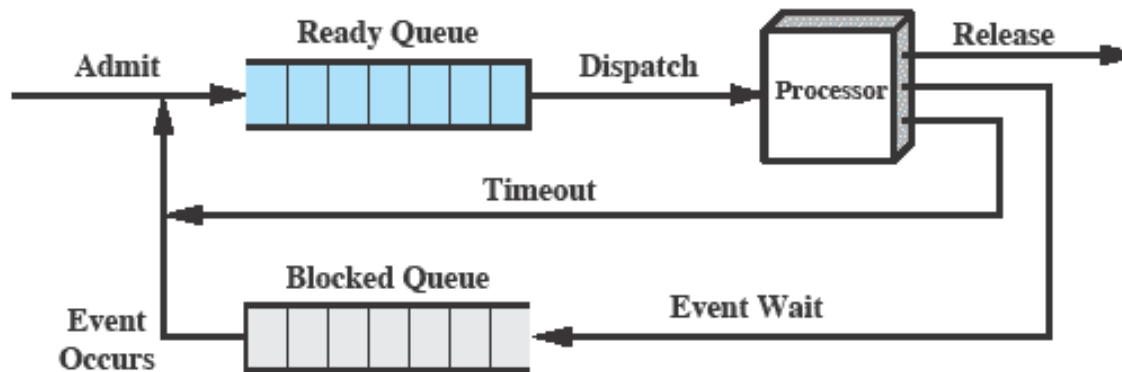
- Modell dafür, wie häufig Prozesse blockieren
- Prozesse verbringen **Anteil p** ihrer Zeit mit Warten auf I/O (also **blocked**)
- Wahrscheinlichkeit dafür, dass **alle n Prozesse blockieren**: p^n
- CPU-Ausnutzung für eine CPU ist dann $1 - p^n$



- Viele I/O-intensive Prozesse (und entsprechend viel Speicher) notwendig, um CPU auszulasten

Warteschlangen zur effizienten Prozessverwaltung

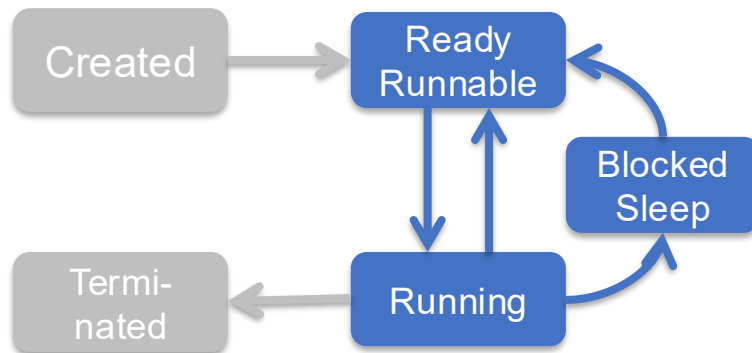
- Effiziente Verwaltung: Prozesse in den Zuständen **Running** oder **Blocked** in **Queues** organisieren
- Bei einem Zustandsmodell mit den 5 Zuständen:



- Viele **verschiedene Gründe für Blockieren** (HDD, Network, ...) erfordern in der Interrupt Service Routine eine Suche nach dem Prozess in Blocked Queue
- Weitere Effizienzsteigerung: je eine **separate Blocked Queue** pro I/O-Event



Linux



```
fuehner@bs-ws25:~$ sudo ps -eo user,pid,ppid,stat,wchan,cmd | grep -v "\["
USER      PID    PPID  STAT  WCHAN  CMD
root         1        0   Ss    do_epo  /sbin/init
root       297        1   Ss    do_epo  /usr/lib/systemd/systemd-journald
systemd+   327        1  Ssl    do_epo  /usr/lib/systemd/systemd-timesyncd
root       356        1   Ss    do_epo  /usr/lib/systemd/systemd-udev
root       604        1   Ss    do_sys  /usr/libexec/bluetooth/bluetoothd
root       605        1   Ss    hrtime  /usr/sbin/cron -f
message+   606        1   Ss    do_epo  /usr/bin/dbus-daemon --system --address=systemd
polkitd    609        1  Ssl    do_sys  /usr/lib/polkit-1/polkitd --no-debug --log-level=debug
root       612        1   Ss    do_epo  /usr/lib/systemd/systemd-logind
avahi      618       603   S      unix_s  avahi-daemon: chroot helper
root       664        1  Ssl    do_sys  /usr/sbin/NetworkManager --no-daemon
root       665        1   Ss    do_sel  /usr/sbin/wpa_supplicant -u -s -O DIR=/run/wpa_supplicant
root       679        1  Ssl    do_sys  /usr/sbin/ModemManager
root       830        1  Ss+    do_sel  /sbin/agetty -o - - \u --noret --noclear - linux
fuehner   9779        1   Ss    do_epo  /usr/lib/systemd/systemd --user
fuehner   9781       9779   S      do_sig  (sd-pam)
fuehner   9799       9779   Ss    do_sys  /usr/bin/mpriproxy
fuehner   9804       9774   S      do_sys  sshd-session: fuehner@pts/0
fuehner   9805       9779   Ss    do_epo  /usr/bin/dbus-daemon --session --address=systemd
fuehner   9806       9804   Ss    do_wai  -bash
root     9825       9806  S+     do_sys  sudo ps -eo user,pid,ppid,stat,wchan,cmd
root     9828       9825  Ss+    do_sys  sudo ps -eo user,pid,ppid,stat,wchan,cmd
root     9829       9828   R      -       ps -eo user,pid,ppid,stat,wchan,cmd
fuehner@bs-ws25:~$
```

- Linux erweitert dieses Grundkonzept um weitere Zustände
- `ps` zeigt Zustand in Spalte **STAT** an
- Ready/Runnable und Running für Anzeige zusammengefasst
- Blocked → Interruptable vs. Uninterruptable Sleep
- `ps` kann sogar den Namen der blockierenden Betriebssystemfunktion anzeigen in Spalte **WCHAN**

Prozesszustände in Linux:

R Runnable

R Running

D Uninterruptable Sleep (Blocked)

S Interruptable Sleep (Blocked)

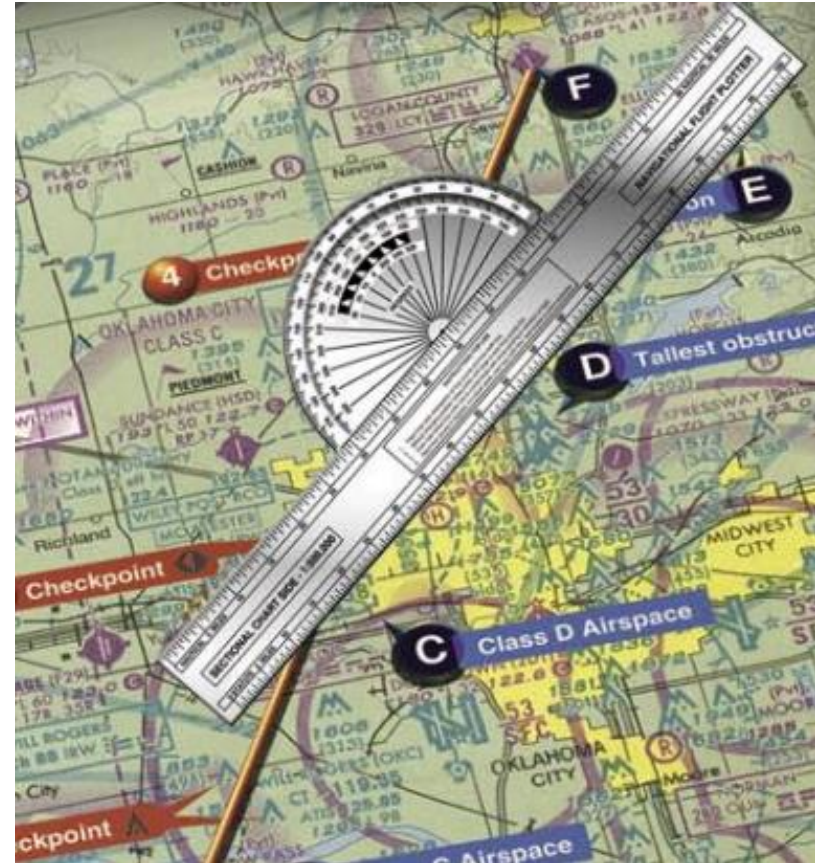
T Terminated/Stopped

Z Zombie (unvollständig beendet)

MacOS?
Windows?

Prozesse

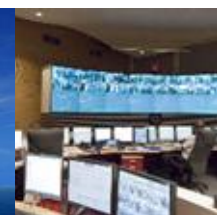
- Prozesse
 - Verwaltung
 - Zustände
 - Dispatching (Umschaltung)
 - Prozesse erzeugen, beenden & unterbrechen
 - Threads
- Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
 - Strategien
 - Metriken und Vergleich



Nur zu Lehrzwecken



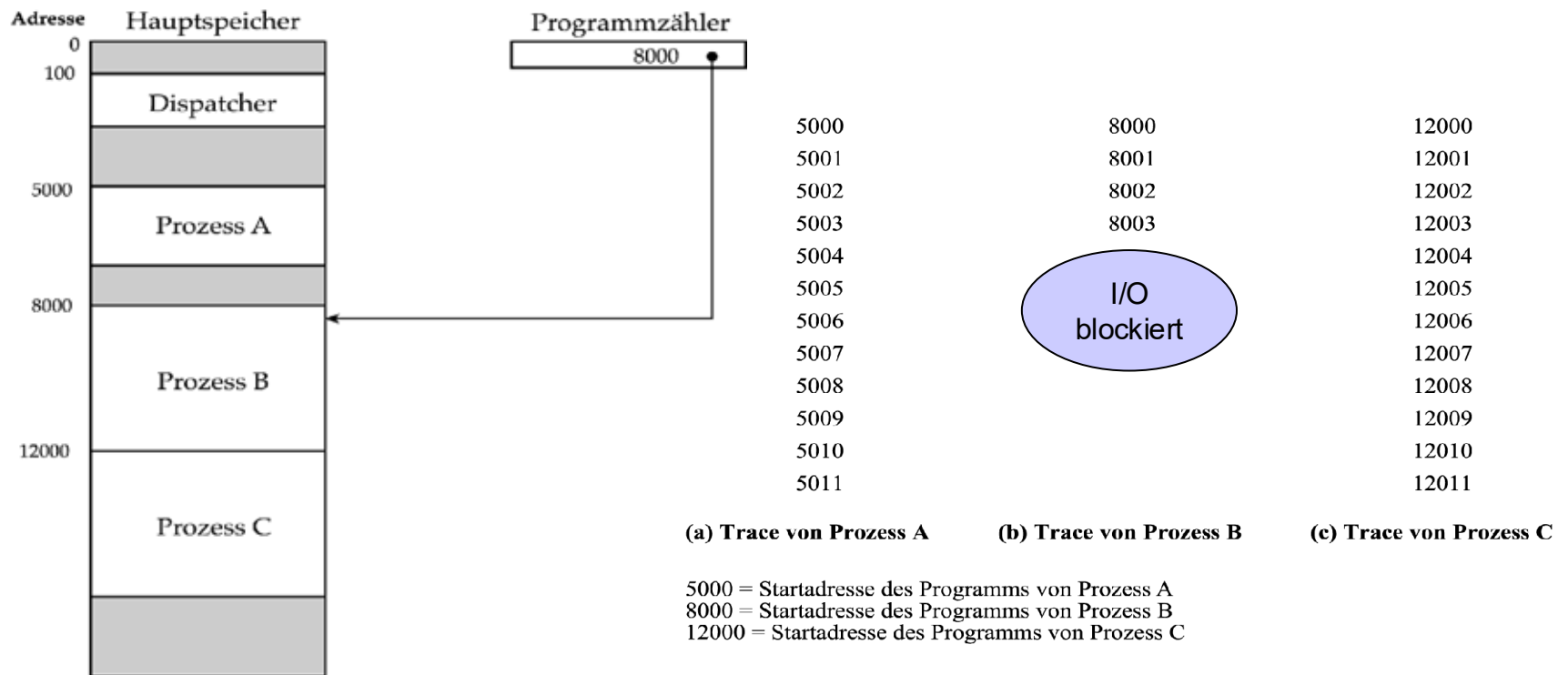
Betriebssysteme



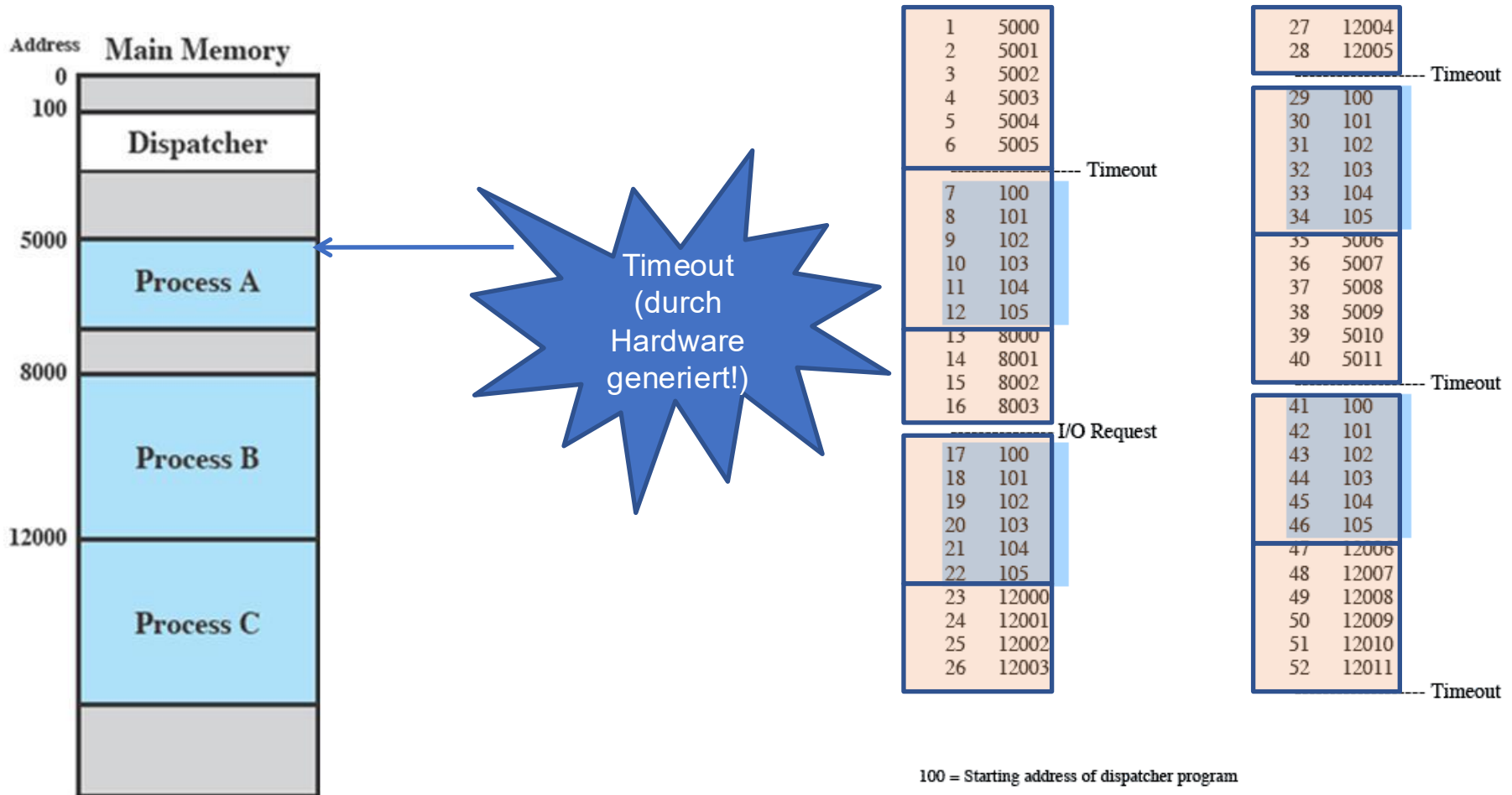
Umschaltung zwischen Prozessen

1. Prozesskontext einschließlich PC und aller Register (z.B. im PCB) sichern
2. Aktualisieren des PCB des aktiven Prozesses
(z. B. State Running → Blocked, warten auf Festplatte)
3. Verschieben des PCB (oder der PID) in die entsprechende Warteschlange
(z. B. Blocked Queue)
4. Scheduling-Entscheidung:
Auswahl eines anderen Prozesses für die Ausführung
5. Aktualisieren des PCB des neuen Prozesses (State → Running)
6. Speicherverwaltungsstrukturen aktualisieren
(→ VL Speicherverwaltung)
7. Prozesskontext des neuen Prozesses wieder herstellen

- **Dispatcher**: Komponente des Betriebssystems, die die CPU zwischen verschiedenen Prozessen umschaltet
- Auswahl des nächsten Prozesses Aufgabe des **Schedulers**



Präemptive Umschaltung von Prozessen



100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

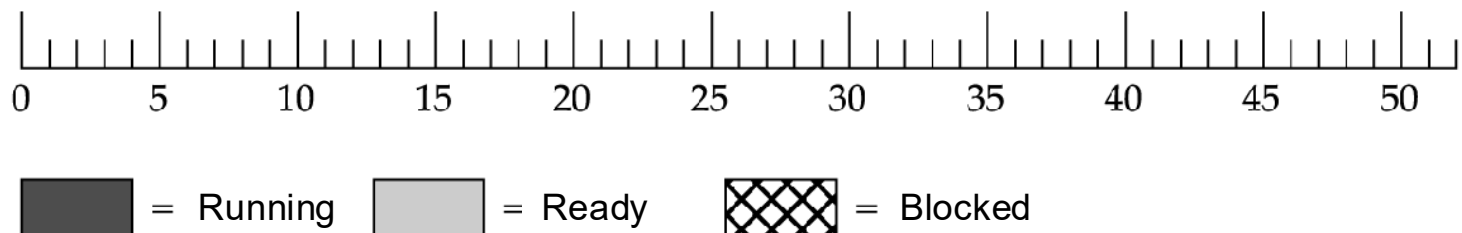
Gantt-Diagramm mit Zuständen der Prozesse über die Zeit

Prozess A

Prozess B

Prozess C

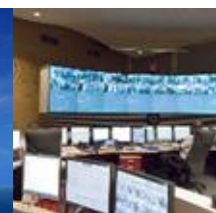
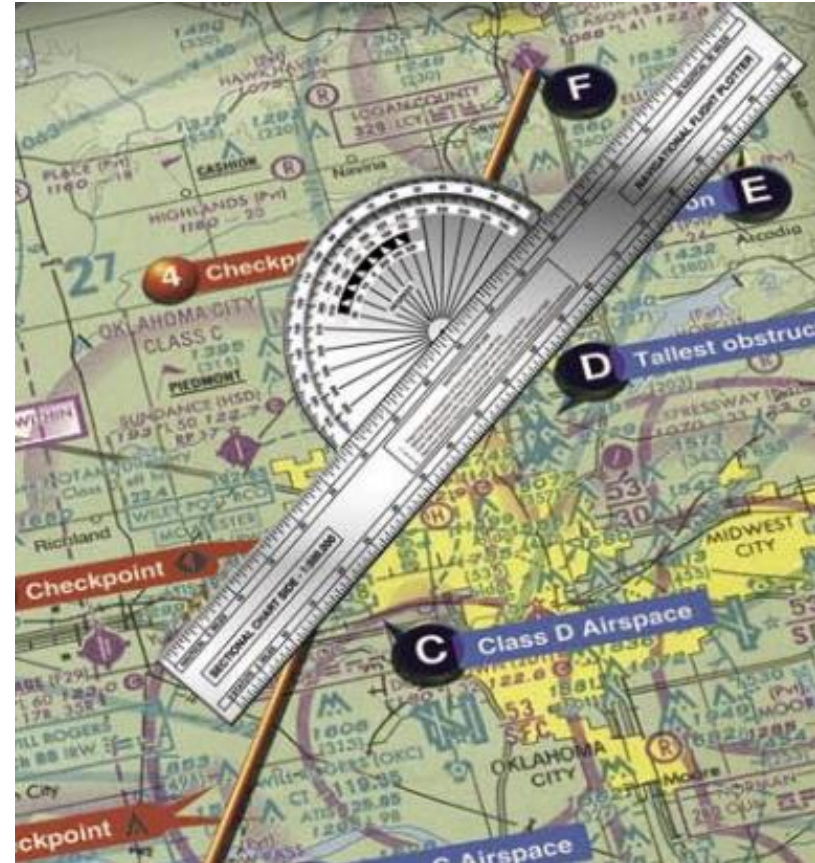
Dispatcher



- Dispatch-Vorgang relativ teuer, aber selten
(Zeit für Kontextwechsel, Reorganisation Seitentabellen, Caches, ... z.B. 1 ms)
- Damit sich das Umschalten lohnt, versucht das Betriebssystem, nur relativ selten umzuschalten (z.B. alle 50 ms)
- **Deshalb Zeit für Dispatching häufig vernachlässigt (→ nicht einzeichnen)**

Prozesse

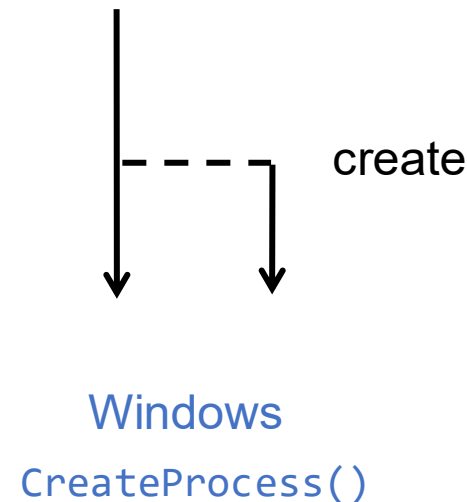
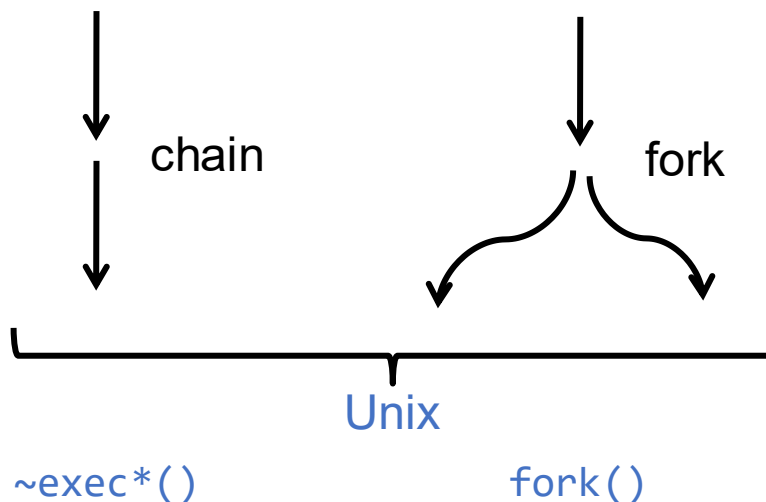
- Prozesse
 - Verwaltung
 - Zustände
 - Dispatching (Umschaltung)
 - Prozesse erzeugen, beenden & unterbrechen
 - Threads
- Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
 - Strategien
 - Metriken und Vergleich





Prozesserzeugung

- **Prozessverkettung (chaining)**: der laufende Prozess startet einen neuen Prozess und terminiert sich damit selbst.
- **Prozessvergabelung (forking)**: Der laufende Prozess startet einen neuen Prozess. Beide Prozesse laufen danach weiter!
- **Prozesserzeugung (creation)**: Der laufende Prozess startet einen unabhängigen neuen Prozess (ähnlich der Prozessvergabelung).

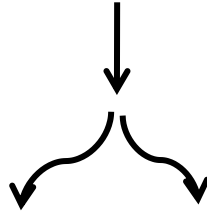


Prozesserzeugung mit fork()

```
#include <unistd.h>
#include <stdio.h>

int main(void) {
    printf("Spass mit fork()\n");
    pid_t ret = fork();

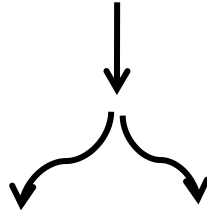
    if (ret < 0) {
        perror("Fehler in fork()");
        return 1;
    } else if (ret == 0) {
        printf("Hallo vom Kindprozess\n");
    } else {
        printf("Elternprozess von Kindprozess %d\n", ret);
    }
    printf("Prozess beenden\n");
    return 0;
}
```



- `fork()` kehrt in beiden Prozessen mit unterschiedlichen Rückgabewerten zurück!

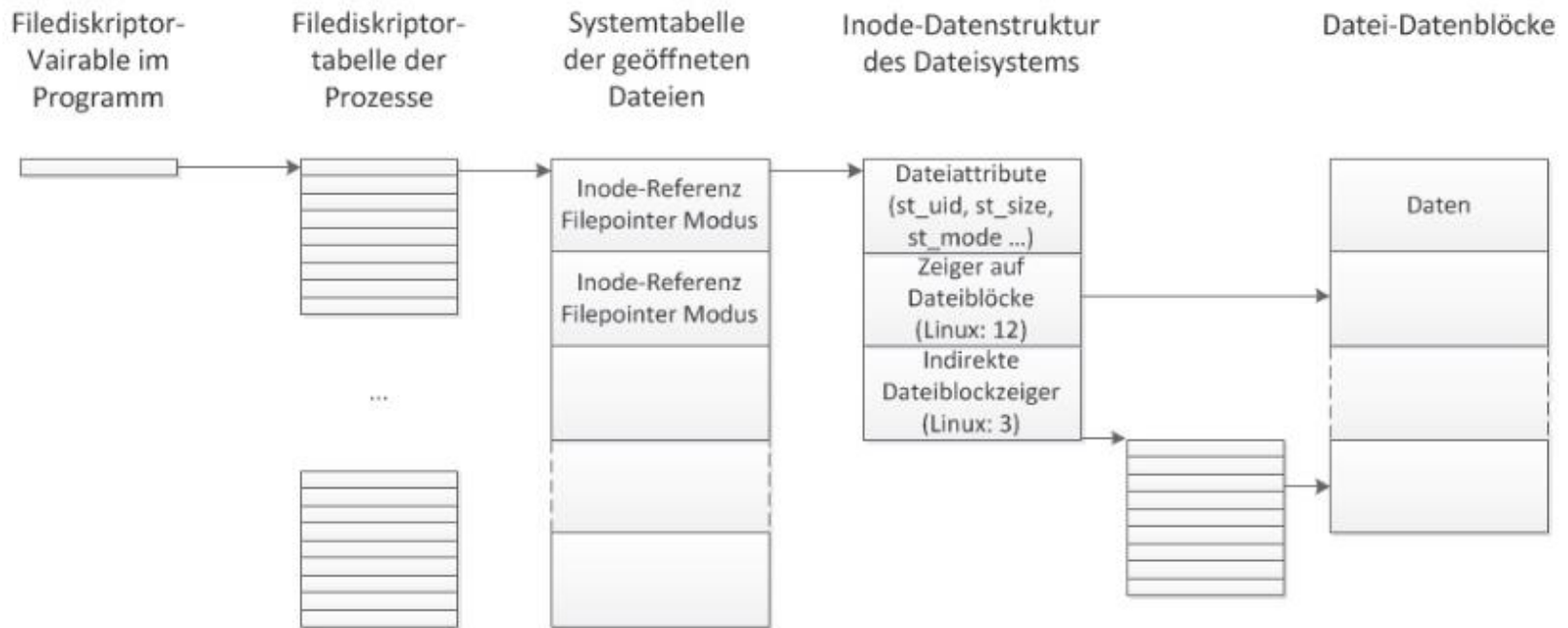
Prozesserzeugung mit fork()

- Systemaufruf `fork()` neuen Prozess als **identische Kopie**
 - aufrufender Prozess heißt **Elternprozess**
 - neu erzeugter Prozess heißt **Kindprozess**
- Programmcode und aktuelle Variableninhalte werden vererbt
 - aber neuer, unabhängiger Adressraum, da neuer, isolierter Prozess
- Geöffnete Dateien werden mit ihren Dateideskriptoren vererbt und geteilt
 - Vor `fork()` geöffnet: Position für Lese-/Schreibzugriff geteilt!
 - Nach `fork()` geöffnet: Dateideskriptoren nur für jeweiligen Prozess
- Implementierung:
 - Kindprozess nutzt meist zunächst den gleichen physikalische Speicher
 - Betriebssystem erzeugt erst bei Schreibzugriff echte Kopie (**copy-on-write**)





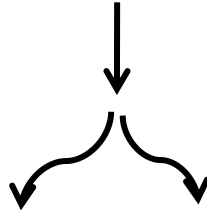
Filedescriptoren bei fork()



- Eine **Filedescriptortabelle** pro Prozess; diese verweist auf **Systemtabelle** für Filedescriptoren, die wiederum auf Inodes und Datenblöcke verweist
- `fork()` kopiert Filedescriptortabelle, aber nicht die Systemtabelle
- Systemtabelle enthält Position für Schreib-/Lesezugriffe in Datei

Prozesshierarchie

- Eltern-Kind-Beziehung führt zu Prozesshierarchie
 - nur ein Elternteil
 - Null oder beliebig viele Kinder
- Eltern und alle Nachkommen bilden eine **Prozessfamilie**
- z.B. mit der Tastatur erzeugte Signale werden an ganze Prozessfamilie verteilt
- **PID/PPID** im PCB speichern die Eltern-Kind-Beziehung
- UNIX: Urvater aller Prozesse ist init
 - beim Booten erzeugt mit PID=1
 - initialisiert das System und ermöglicht Login
- Windows: Prozesse bilden keine Hierarchie (einfache Liste)

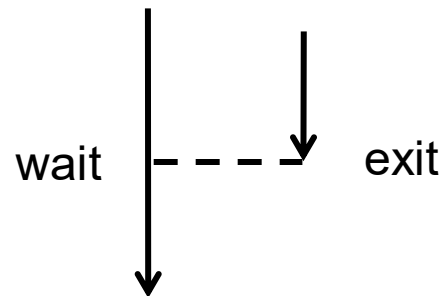
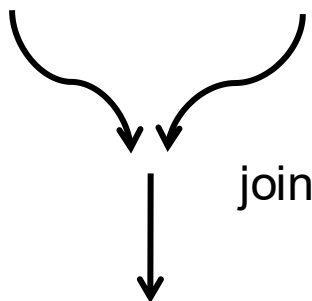




Prozessterminierung

Prozesszusammenführung relativ ähnlich, in jedem Fall wird der erzeugte Prozess terminiert:

- **Prozessvereinigung (join):** der erzeugende Prozess wartet auf den erzeugten Prozess
- **Prozesstreffen (wait):** der erzeugende Prozess *kann* auf den erzeugten Prozess warten (verschiedene Wait-Varianten).



Unix

Prozessterminierung

- Aktuellen Prozess terminieren:

```
#include <unistd.h>

void _exit(int status);
```

- Ähnliche C-Bibliotheksfunktion `exit()` nutzt intern `_exit()`
- Beenden der `main()`-Funktion (mit oder ohne `return`) ruft automatisch `exit()` auf
- Argument `status` wird als **Exit Code** an den Elternprozess übergeben (nur ein Byte ohne Vorzeichen, `uint8_t`)
- Neben dem freiwilligen Beenden können Prozesse auch unfreiwillig
 - durch andere Prozesse (`kill -9`) oder
 - bei schweren Fehlern wie fehlerhaften Speicherzugriffen oder Division durch 0 durch das Betriebssystem beendet werden!

Prozessterminierung

- Prozess blockieren, bis ein Kind den Status ändert (endet / angehalten wird):

```
#include <sys/wait.h>

pid_t wait(int *status);
```

- Exit Code kann in `status` abgefragt werden
- Beendete Kindprozesse werden zu **Zombies**, bis ihre Eltern den Exit Code lesen (`wait()` / `waitpid()` / Signal-Handler für `SIGCHLD`)
- **Zombies** belegen PCB-Einträge (vollaufen!), aber keine CPU oder Speicher
- **Waisenprozesse (Orphans)**
 - Elternprozess endet vor Kindprozess → Kind wird zum Waisen
 - `init`- bzw. `systemd`-Prozess (PID 1) adoptiert alle Waisen (→ PPID=1)
 - Waisen werden beim Beenden nicht zu **Zombies**

Signale

- Signale sind **asynchrone Benachrichtigungen** an einen Prozess
- Jedes Signal hat eine Nummer (und zugeordneten Namen)
- Typische Reaktionen: Ignorieren, Terminieren, eigener Singalhandler
- Signale senden mit

```
#include <signal.h>
int kill(pid_t pid, int sig); // 0=ok
```

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void handler(int sig) {
    printf("Signal %d!\n", sig);
}

int main() {
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    // moderner: sigaction()
    pause(); // Pause bis Signal ...
    return 0; // ... dann beenden
}
```

Signal	Erklärung
SIGHUP (1)	Hang Up – Terminal geschlossen (z.B. ssh-Verbindung, Ausloggen)
SIGINT (2)	Tastatur-Interrupt (Ctrl-C)
SIGCHLD (17?)	Kindprozess hat Status geändert (beendet oder angehalten)
SIGKILL (9)	Sofortiges Beenden – nicht abfangbar!
SIGTERM (15)	Höfliche Bitte um Beendigung

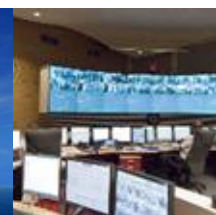
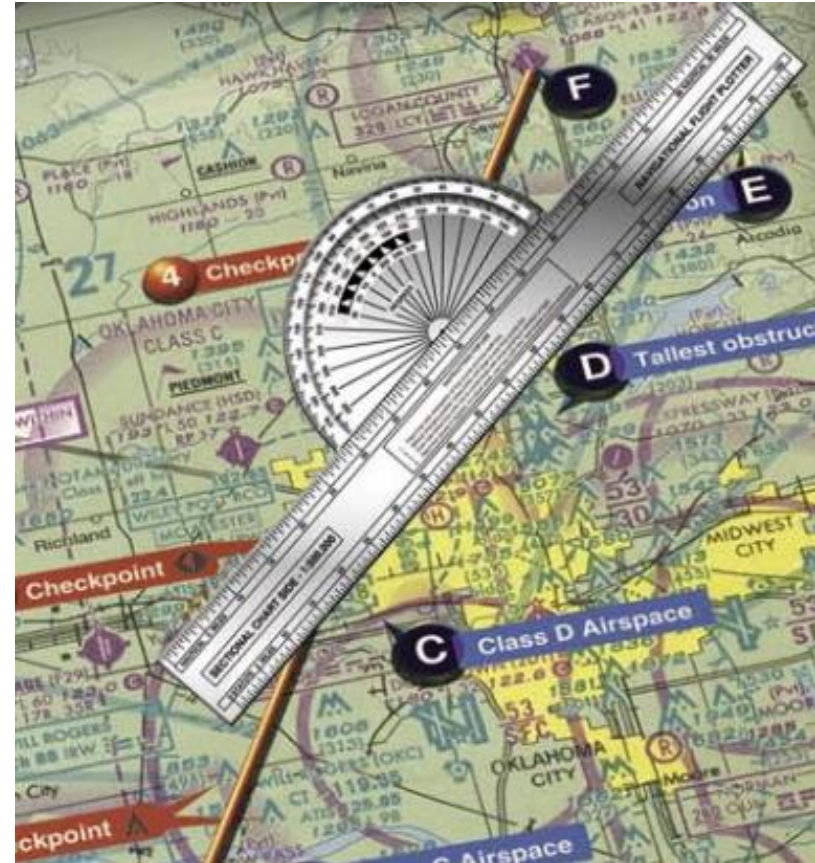
fork() – exec() – wait()

Was ist `exec*()`?

- Ganze Familie von Funktionsaufrufen mit unterschiedlichen Parametern
- Kehrt bei Erfolg nicht zurück
- Programm wird durch neues Programm ersetzt
- PID bleibt erhalten
- Geöffnete Filedescriptoren bleiben erhalten
- Drei Schritte, um andere Programme aufzurufen:
 1. `fork()` Kindprozess als Kopie des Elternprozesses erzeugen
 2. `exec*()` Im Kindprozess neues Programm laden (überschreibt Speicher)
 3. `wait()` Elternprozess wartet auf Terminierung des Kindes

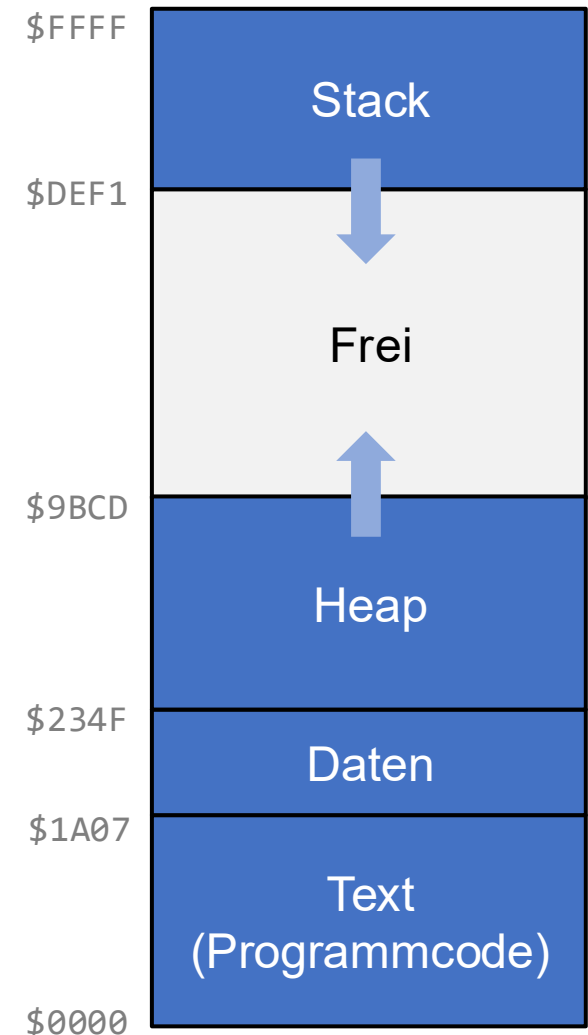
Prozesse

- Prozesse
 - Verwaltung
 - Zustände
 - Dispatching (Umschaltung)
 - Prozesse erzeugen, beenden & unterbrechen
 - Threads
- Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
 - Strategien
 - Metriken und Vergleich



Leichtgewichtige Alternative zu Prozessen

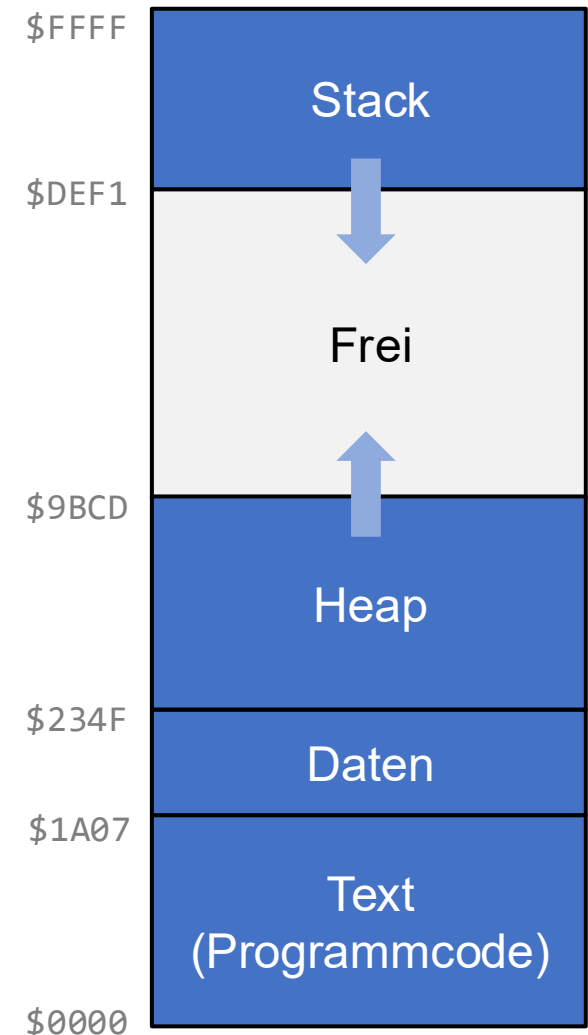
- Ein Thread ist ein Ausführungsweg (Ausführungspfad) **innerhalb** eines Prozesses
- Threads haben **keine eigenen Ressourcen** (Speicher, offene Dateien, ...)
- Bei **enger Kooperation** innerhalb eines Programms sind Threads gegenüber neuen Prozessen von Vorteil
 - Speicherisolation nicht notwendig
 - einfachere Kommunikation
 - weniger Overhead (Faktor 10x-100x)



Leichtgewichtige Alternative zu Prozessen

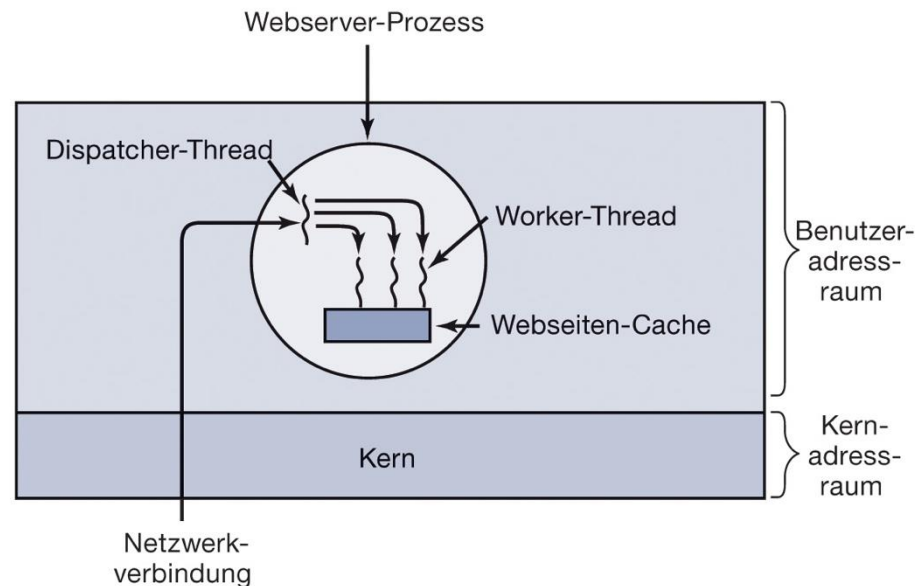
Ein Thread enthält folgende Elemente:

- **Zustand:** aktiv, bereit, ... (meist wie Prozess)
- **Kontext:** PC (Programmzähler), SP (Stackpointer), Framepointer etc.
- **Stack:** Eigene lokale Variablen und Rücksprungadressen für Funktionsaufrufe
- Speicherplatz für Thread-lokale Variablen
- Innerhalb eines Prozesses hat **jeder Thread Zugriff auf gemeinsame Ressourcen** (Speicher, offene Dateien, ...)



Nutzungsbeispiel: Web Server

- Serverprozess soll Webseiten ausliefern
- Caching stark frequentierter Seiten im RAM



- **Dispatcher-Thread** liest über das Netzwerk eingehende Request
- Request wird dann an **Worker-Thread** aus einem Pool übergeben
- **Worker-Thread** liest Daten von Disk, wenn nicht im Cache
- Cache wird von allen Worker-Threads geteilt

Nutzungsbeispiel: Web Server

- Serverprozess soll Webseiten ausliefern
- Caching stark frequentierter Seiten im RAM

Dispatcher-Thread

```
while (TRUE) {  
    get_next_request(&buf);  
    handoff_work(&buf);  
}
```

Worker-Thread

```
while (TRUE) {  
    wait_for_work(&buf)  
    look_for_page_in_cache(&buf, &page);  
    if (page_not_in_cache(&page))  
        read_page_from_disk(&buf, &page);  
    return_page(&page);  
}
```

- **Dispatcher-Thread** liest über das Netzwerk eingehende Request
- Request wird dann an **Worker-Thread** aus einem Pool übergeben
- **Worker-Thread** liest Daten von Disk, wenn nicht im Cache
- Cache wird von allen Worker-Threads geteilt

Implementierungsvariante User Space Threads

Definition: Threads, die ohne Unterstützung des Betriebssystemkerns vollständig im User Space verwaltet werden

- Kernel **kennt nur den Prozess**, nicht die einzelnen Threads
- Implementierung über Bibliothek (z.B. POSIX Pthreads, Green Threads)
- Bibliothek verwaltet Thread-Tabelle wird im User Space gehalten (je Prozess)
- Scheduling erfolgt durch die Bibliothek, nicht durch den Kernel
- Vorteile:
 - Sehr schneller Thread-Wechsel ohne Kernel-Beteiligung
 - Nachträgliche Implementierung unabhängig vom verwendeten Betriebssystem möglich
- Aber:
 - Nur **kooperativer** Thread-Wechsel (**yield()**-Funktion)
 - Blockierender Syscall **blockiert alle Threads** des Prozesses
 - Kein echtes Multiprocessing (nur ein CPU-Kern nutzbar)

Implementierungsvariante: Kernel Space Threads

Definition: Threads, die vom Betriebssystem-Kernel verwaltet und gescheduled werden

- Der Kernel kennt jeden einzelnen Thread und verwaltet diese direkt
- Thread-Tabelle wird im Kernel Space gehalten
- Erzeugung und Zerstörung von Threads über Systemaufrufe (Syscalls)
- Scheduling erfolgt durch den Kernel-Scheduler
- Vorteile:
 - Echtes Multiprocessing möglich (Threads auf mehrere CPU-Kerne verteilbar)
 - Blockierender Syscall blockiert nur den aufrufenden Thread, nicht den ganzen Prozess
 - Präemptiver Thread-Wechsel ohne Kooperation möglich
- Nachteile:
 - Thread-Wechsel erfordert Kontextwechsel in den Kernel (langsamer)
 - Höherer Overhead bei Erzeugung und Verwaltung von Threads

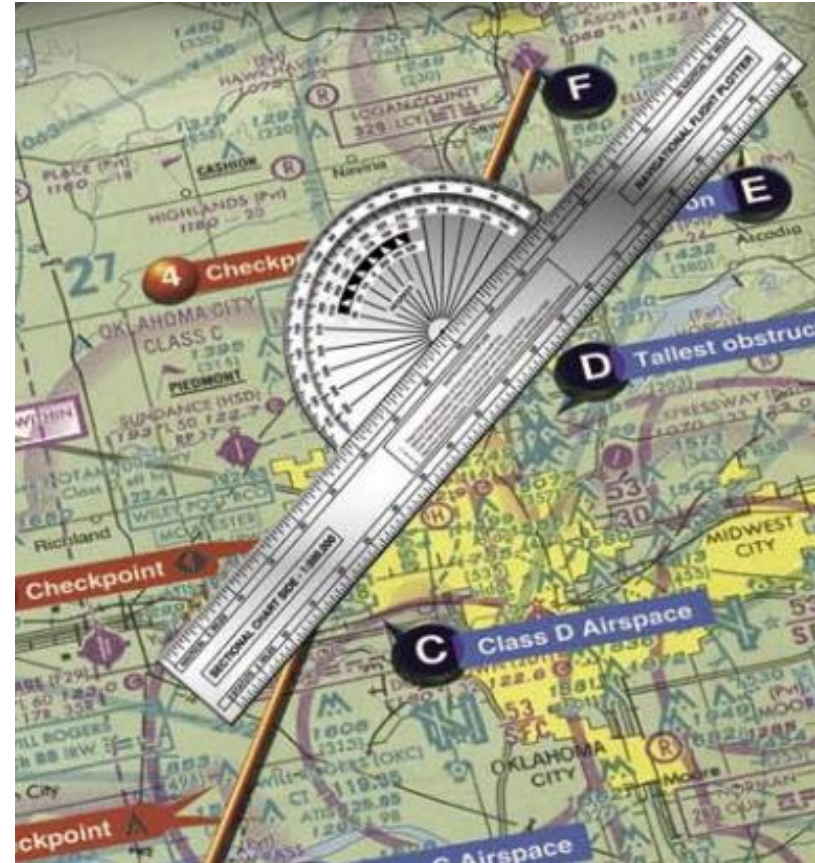
POSIX Threads: Lebenszyklus managen

```
#include <pthread.h>
int pthread_create(pthread_t *thread,
    const pthread_attr_t *attr,
    void *(*start_routine)(void *),
    void *arg);
```

- thread wird als Rückgabewert verwendet und dient für spätere Aufrufe als Handle
- attr kann NULL sein oder Attribute enthalten
- start_routine ist eine reguläre Funktion, die im neuen Thread gestartet wird
- Enthält den Pointer arg als Übergabeparameter
- Beenden des Threads mit regulärem return oder pthread_exit(void *retval) oder pthread_cancel(pthread_t thread)
- Auf Beendigung eines Threads und Rückgabewert warten mit pthread_join(pthread_t thread, void **retval) (ähnlich waitpid())
- Threads können Rechenzeit abgeben mit blockierendem Aufruf und werden vom Scheduler ansonsten wie Prozesse behandelt

Prozesse

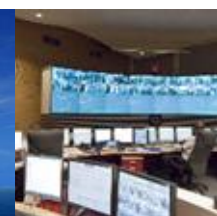
- Prozesse
- ➔ Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
 - Strategien
 - First Come First Serve
 - Shortest Process Next
 - Round Robin
 - Statische Prioritäten
 - Multilevel Feedback
 - Metriken und Vergleich



Nur zu Lehrzwecken



Betriebssysteme



Einführung

- **Scheduler** wählt den Prozess aus, der als nächster laufen soll
- Scheduler ist Teil des Betriebssystems
- Eigentliche Umschaltung durch **Dispatcher**
- Scheduling-Entscheidung wird interessant, wenn mehrere Prozesse **gleichzeitig Ready/Runnable** sind
- Scheduling-Entscheidung in **Mehrkern- und Multiprozessorsystemen pro Kern treffen** – in dieser Veranstaltung meist nur ein Kern betrachtet
- Ähnliche Entscheidung z.B. auch für I/O zu treffen, z.B. HDD I/O Scheduling

Einführung

- Viele verschiedene Scheduling-Algorithmen und -Strategien
- Kategorien von Strategien nach Einsatzgebiet:
 1. **Stapelverarbeitung** – Gehaltsabrechnungen, Zinsgutschriften, KI-Lernen
(Keine Interaktion mit ungeduldig wartenden Benutzern, eher Serveranw.)
 2. **Interaktive Systeme** – Textverarbeitung, IDE, Hotelbuchung
(betrifft auch Serveranwendungen z.B. Buchungssysteme)
 3. **Echtzeitsysteme** – Steuerung einer Industrieanlage, Autopilot im Flugzeug
*(Sonderfall eingebettete Systeme und Audio/Video:
nur vorab bekannte Prozesse und Abläufe, kein Allzweckgebrauch)*

Kriterien und Ziele

- **Fairness** – jeder Prozess bekommt einen fairen Anteil der CPU-Zeit
- **Balance** – alle Teile des Systems sind ausgelastet
- **Durchsatz** – Maximierung der Jobs pro Zeiteinheit
- **Durchlaufzeit** – Minimierung der Zeit vom Start bis zur Beendigung
- **CPU-Ausnutzung** – die CPU ist immer ausgelastet
- **Antwortzeit** – schnelle Reaktion auf Anfragen
- **Proportionalität** – Erwartung Benutzer an Aufwand
- **Deadlines** – Klar definierte Deadlines einhalten
- **Vorhersagbarkeit** – vorhersehbare Reaktionszeit auf Anfragen

→ Ziele stehen zum Teil im Konflikt, Priorisierung abhängig von Systemtyp wie: Desktop, Server, Handy, Waschmaschine, autonomes Fahrzeug, Airbus A350, ...

Wann schedulen?

- Wann Scheduling-Entscheidung (mit eventuellem Dispatching eines neuen Prozesses) treffen?
 - 1. Erzeugung Kindprozess**
Eltern-, Kind- oder dritten Prozess fortsetzen?
 - 2. Terminierung eines Prozesses**
Welcher andere Prozess soll weiterrechnen?
 - 3. Blockierender Systemaufruf**
Welcher andere Prozess soll weiterrechnen?
 - 4. Optional bei Aufhebung der Blockierung eines Prozesses**
Sofortiges Dispatching des blockierten Prozesses z.B. bei I/O Interrupt ermöglichen?
 - 5. Timerinterrupt (z.B. 50 Hz)**
Regelmäßiges Dispatching von Prozessen in Ready ermöglichen?

Varianten

Kooperative Prozessorvergabe

- Umschaltung erfolgt nur, wenn Prozess **freiwillig** den Prozessor abgibt oder **terminiert**
- Fast alle Systeme schalten auch um, wenn ein Systemaufruf **blockieren** würde
- CPU-intensiver Prozess kann so **stundenlang** laufen, wenn er nicht kooperiert
- **Reaktion anderer Prozesse** auf externe Ereignisse entsprechend verspätet
- Wieder populär durch **async/await** in **Python** (Sprachfeature + asyncio), **Node.js** (Run-To-Completion Microtasks in Engine + libuv), Java Virtual Threads!

Präemptive Prozessorvergabe

- Umschaltung erfolgt durch (Timer-)interrupts
- Wenig Beeinflussung der Prozesse untereinander
- Wird verwendet, wenn Prozesse eine lange Laufzeit haben, oder die Reaktionszeit auf externe Ereignisse kurz sein muss

Einordnung User Space / Kernel Space Threads?

Wdh.: Wie häufig blockieren Prozesse?

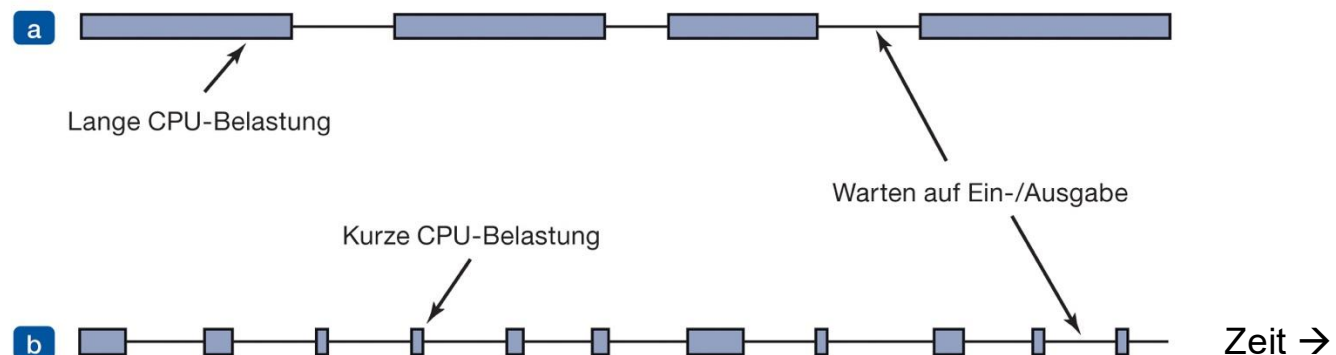
CPU-intensive Prozesse (CPU-lastig, CPU bound)

- lange CPU-Nutzungszeiten und selten Ein-/Ausgabe-Wartezeiten.

E/A-intensive Prozesse (E/A-lastig, I/O bound)

- häufige E-/A-Wartezeiten (z.B. durch die Festplatte)
- kurze CPU-Nutzungszeiten

Beispiel



- Scheduling-Verfahren können bestimmte Workload-Typen bevorzugen!

Prozessoraffinität auf Multicore- / Multiprozessorsystemen

Problem für Multiprozessor- und Multicoresysteme

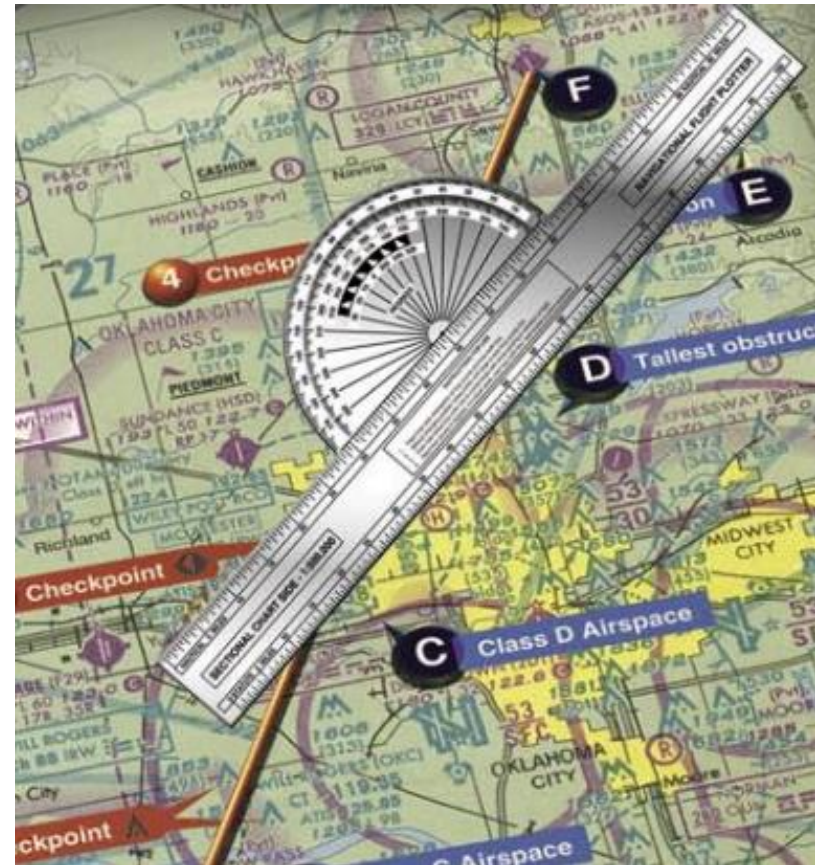
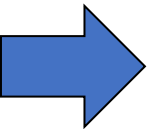
- Schnellste CPU-Caches sind nur lokal für einen einzigen Kern
- Bei Scheduling desselben Prozesses auf einem neuen Kern
 - ➔ keine relevanten Daten im Cache
 - ➔ Cache Miss kostet viel Zeit (Neuladen + evtl. Zurückschreiben)

Lösung

- (implizite) Prozessoraffinität:
Kernel versucht, einem Prozess immer den gleichen Kern zuzuweisen
- wird z.B. in Linux und Windows verwendet
- Linux unterstützt auch explizite Prozessoraffinität
(Kern per System Call setzen, erfordert Root-Rechte)

Prozesse

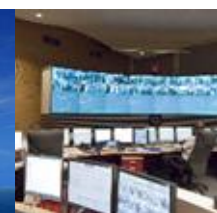
- Prozesse
- Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
- Strategien
 - First Come First Serve
 - Shortest Process Next
 - Round Robin
 - Statische Prioritäten
 - Multilevel Feedback
 - Metriken und Vergleich



Nur zu Lehrzwecken



Betriebssysteme



- **First Come First Serve (FCFS)**

- Rechenbereite Prozesse werden in eine Warteschlange eingeordnet und gemäß ihrer *Ankunftszeit* von der CPU abgearbeitet.
- Blockiert ein Prozess, dann kommt der nächste Prozess an die Reihe. Wird der blockierte Prozess wieder rechenbereit, wird er neu am Ende der Warteschlange einsortiert.
- Vorteil: einfach
- Nachteil: hohe Durchlaufzeit von E/A-intensiven Prozessen

- **Shortest Process Next (SPN)**

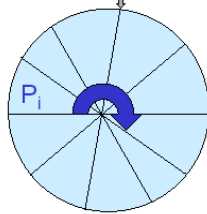
Meist nicht-präemptiv...
präemptiv als Shortest Remaining Time First (SRTF)

- Die bereiten Prozesse werden gemäß ihrer **ursprünglich geplanten Laufzeit** (Service Time) in der Warteschlange sortiert und abgearbeitet

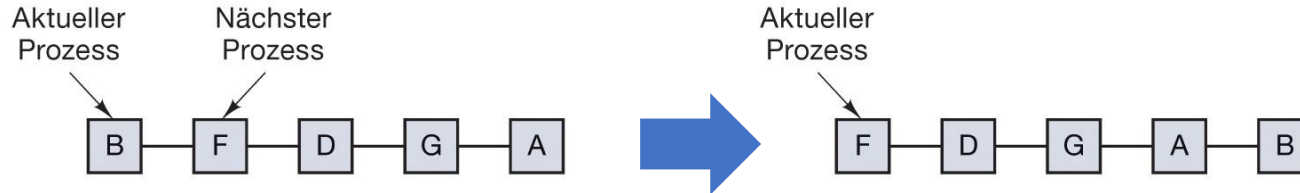


- Bevorzugt kurze Prozesse (z.B. E/A-intensive, wenn Laufzeit nur CPU-Zeit berücksichtigt)
- Kenntnis über Laufzeit aus Erfahrung oder Historie

Round Robin (RR)



- Jeder bereite Prozess darf eine gewisse Zeit Z laufen, wird dann unterbrochen (nur präemptiv), und dann an das Ende der Warteschlange gestellt.

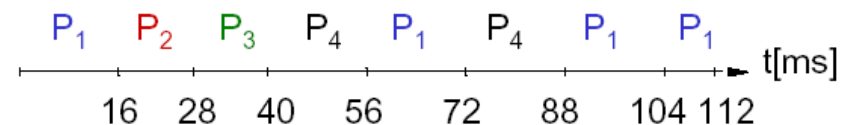


- Zeitscheibe (Timeslice):** feste Zeitspanne Z (Quantum), typisch Z 1-100 ms
- Dauer der Zeitscheibe zu kurz $\rightarrow$ hoher Dispatching-Overhead
- ... zu lang $\rightarrow$ lange Wartezeit für interaktive Prozesse, die schnell blockieren
- RR ist unfair zu I/O intensiven Prozessen (Zeitscheiben häufig nicht ausgenutzt, da die Prozesse schon davor durch I/O blockieren)
- Bei Blockieren oder Terminieren eines Prozesses endet die Zeitscheibe vorzeitig

Beispiel:

Prozess	benötigte CPU-Zeit
P_1	56 ms
P_2	12 ms
P_3	12 ms
P_4	32 ms

$\Rightarrow$ Schedule (Ablauf) für $Z = 16$ ms



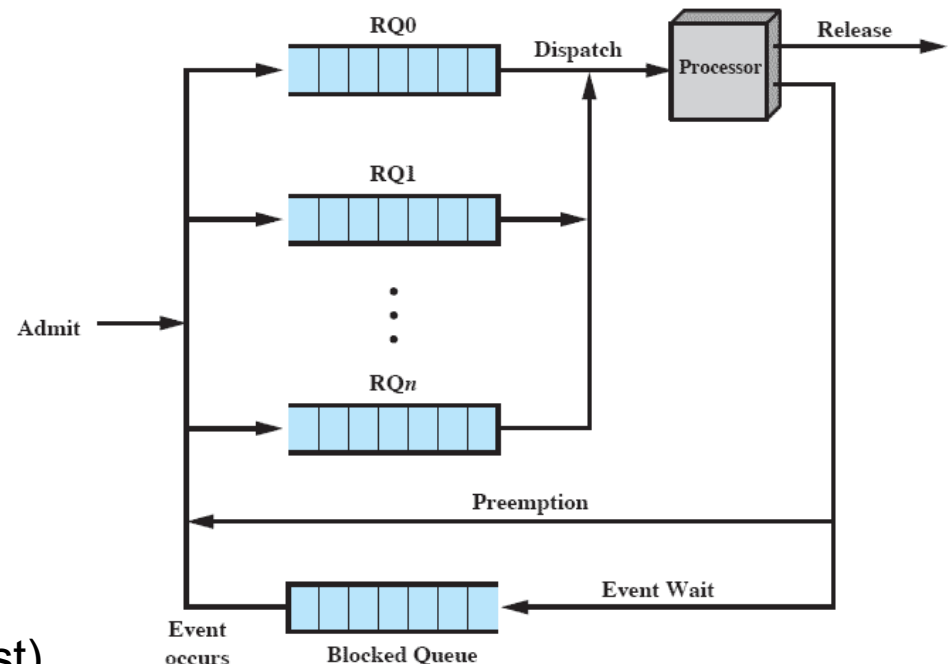
Statische Prioritäten

- Jeder Prozess erhält eine **eindeutige Priorität**
- Priorität ist über die Lebensspanne des Prozesses **konstant (statisch)**
- **Scheduler wählt immer den Prozess mit der höchsten Priorität**
- Ein Prozess erhält also nur Rechenzeit, wenn alle höherprioren Prozesse im Zustand **Blocked** (oder **Terminated**) sind!
- Mögliches Problem: Starvation (ein Prozess wird nie ausgewählt)

- Wichtiges Verfahren für garantierte Antwortzeiten in Echtzeitsystemen (z. B. Airbag oder ABS im Auto oder Flugsteuerrechner im Airbus)
- Statisch priorisierte Scheduler für Desktop und Server ungeeignet

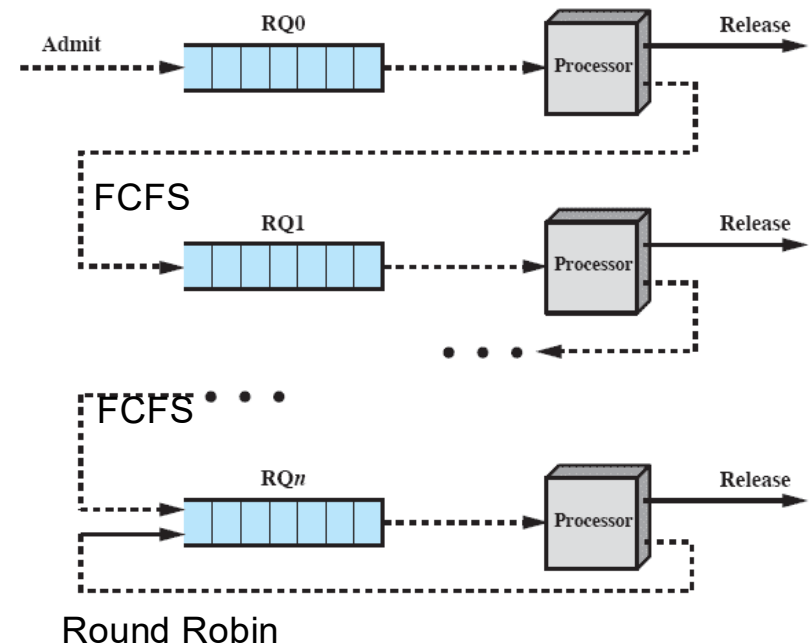
Multi-Level Scheduling (ML)

- Jedem Prozess wird eine **Priorität (Wichtigkeitsstufe)** durch den Anwender oder ein gesondertes Verfahren zugeordnet.
- Es läuft immer nur der **bereite Prozess (ready)** mit der **höchsten Priorität**; laufende Prozesse mit **geringerer Priorität** werden **verdrängt**.
- Bei gleicher Prioritätsstufe meist Round-Robin oder FCFS
- Prioritäten können statisch oder dynamisch vergeben werden
- Auch hier Starvation möglich (Prozess wird nie ausgewählt)
- Die Umsetzung erfolgt mit
 - mehreren Ready Queues für unterschiedliche Prioritäten (bei geringer Anzahl von Prios)
 - geordneter Ready Queue (sonst)



Multi-Level Feedback (MLFB)

- Ein Prozess wird abhängig von seiner *bisherigen* Laufzeit priorisiert.
- Wird ein Prozess neu erzeugt, dann erhält er die **höchste Priorität**. Wenn dieser Prozess das erste Mal beendet wird, dann wird auch seine Priorität **schrittweise herabgestuft** (siehe Abbildung).
- Präemptiv mit festem Zeitintervall
- Starvation möglich: lang laufende Prozesse werden immer weiter herunter priorisiert
- Lösung:
 - Größere Zeitintervalle für Herabstufung je niedriger die Priorität
 - herabgestufte Prozesse allmählich wieder höher priorisieren (-> Unix-Scheduler)



- Multilevel-Feedback, präemptiv mit festem **Zeitintervall** für Repriorisierung
- Die **Priorität eines Prozesses** wird abhängig z.B. von seiner verbrauchten CPU-Zeit **periodisch berechnet**. Für Schritt i :

$$T_{CPU}(i) = \frac{T_{CPU}(i-1)}{2}; \quad P(i) = P_{base} + P_{nice} + \frac{T_{CPU}(i)}{2}$$

P : Priorität (kleinere Werte bedeuten höhere Priorität)

P_{base} : Basispriorität (wird vom System bestimmt)

P_{nice} : Nice-Level (kann vom Benutzer angepasst werden)

T_{CPU} : verbrauchte CPU-Zeit in der Einheit „Ticks“ (z.B. 10 ms);
wird um die Zeit inkrementiert, die der Prozess rechnet
und nach jedem **Zeitintervall** mit obiger Formel neu bestimmt

i : Nummer des Scheduling-Intervalls

- Prozesse werden über die Basispriorität unterschiedlichen **Bändern** (Swapper, I/O Geräte, ..., Benutzerprozesse) zugeordnet, ein Wechsel ist *nicht* möglich.
- Neuberechnung der Priorität erfolgt nur für Benutzerprozesse.

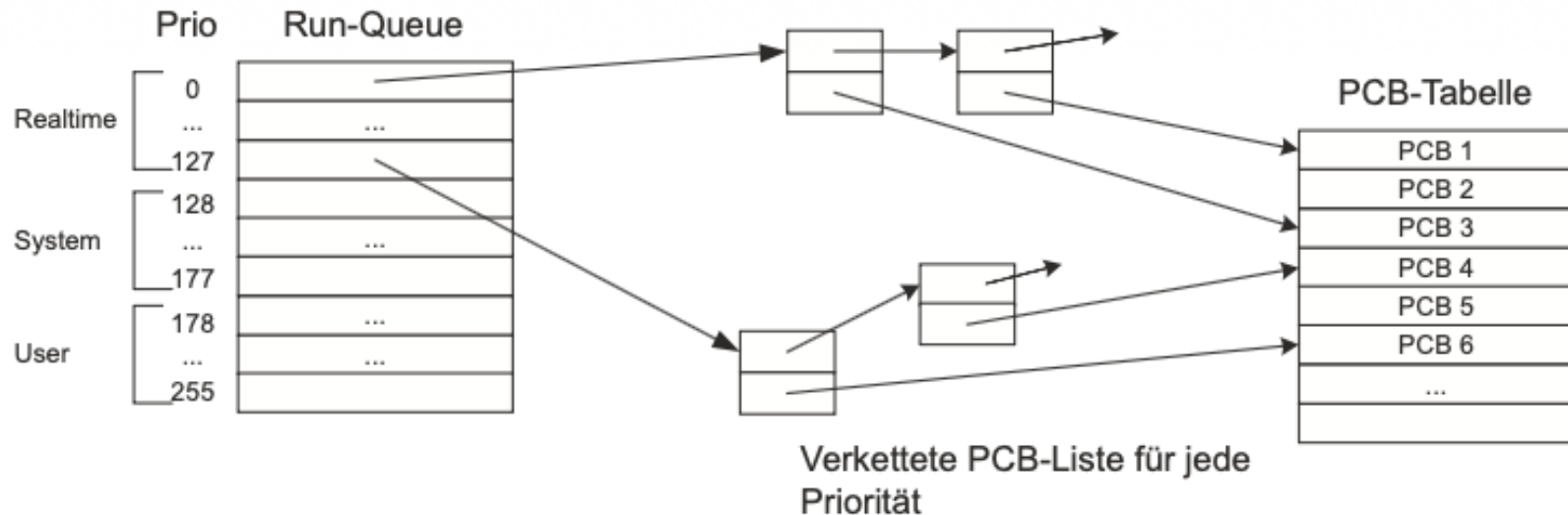


Time	Process A		Process B		Process C	
	Priority	CPU count	Priority	CPU count	Priority	CPU count
0	60	0 1 2 ...	60	0	60	0
1	75	30	60 0 1 2 ...	0	60	0
2	67	15	75	30	60 0 1 2 ...	0
3	63	7 8 9 ...	67	15	75 30	0
4	76	33	63 7 8 9 ...	7	67	15
5	68	16	76	33	63	7

Priority = $P(i)$
CPU count = $T_{CPU}(i)$

- Gegeben seien 3 Benutzerprozesse A, B, C
- Ein Timer-Interrupt unterbricht das System 60 mal pro Sekunde und zählt für den laufenden Prozess einen Zähler (**CPU Count** T_{CPU}) hoch.
- Jede Sekunde (Zeitintervall) werden die Werte von T_{CPU} halbiert und anschließend die **Prioritäten neu berechnet**.
- Auf Grundlage der Prioritäten wird dann eine **neue Scheduling-entscheidung** getroffen.
- Ergebnis: Prozess wird nach einem Wechsel stark herunter priorisiert und steigt dann langsam wieder auf.

Typische Umsetzung



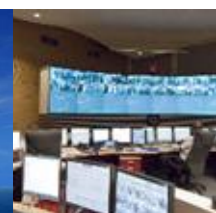
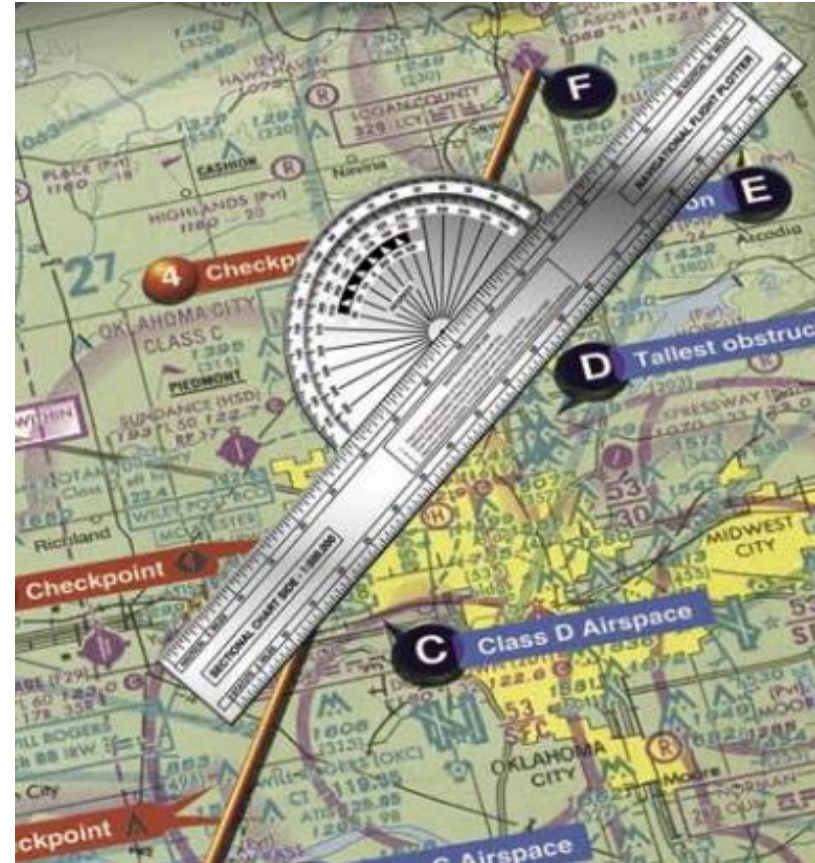
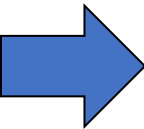
- Run-Queue ist die zentrale Datenstruktur des Schedulers im Kernel
- Array, dass für jede Priorität eine verkettete Liste mit Zeigern auf PCBs enthält
- Scheduler im Linux-Kernel wurde häufiger überarbeitet:
Klassischer Unix-Scheduler -> O(1)-Scheduler -> CFS -> EEVDF

Completely Fair Scheduler (CFS)

- Ziel: *faire* Verteilung der CPU-Zeit auf alle lauffähigen Prozesse
- Keine festen Zeitscheiben – Scheduling-Granularität passt sich dynamisch an
- Standard-Scheduler im Linux-Kernel (Kernel Version 2.6.23 bis 6.6.0)
- Für jeden Prozess **Virtual Runtime (vruntime)** mit bisher verbrauchter CPU-Zeit messen (blockierte Zeit zählt nicht mit)
- Prozess mit **niedrigster vruntime** als nächsten auswählen
Rot-Schwarz-Baum nach vruntime, Knoten ganz links cachern
- Außerdem **min_vruntime** mit Minimum der aktuell lauffähigen Prozesse
- Lange blockierte Prozesse nicht zu sehr bevorteilen: Beim Übergang aus blocked **$vruntime = \max(vruntime, min_vruntime)$** setzen
- Ältere Prozesse nicht schlechter zu stellen: neue Prozesse mit **$vruntime = min_vruntime$** starten
- Nice-Werte führen zu Über-/Untergewichtung der **vruntime**-Inkmente

Prozesse

- Prozesse
- Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
 - Strategien
 - First Come First Serve
 - Shortest Process Next
 - Round Robin
 - Statische Prioritäten
 - Multilevel Feedback
 - Metriken und Vergleich



Prozesse werden beschrieben durch

- **Arrival Time T_a** – Zeitpunkt zu dem der Prozess erstmalig *bereit* wird
- **Service Time T_s** – Ausführungsdauer (Rechenzeit)

Einfacher Vergleich von Verfahren durch

- **Completion Time T_c** – Zeitpunkt, zu dem ein Prozess beendet wird.
- **Turnaround Time T_r** – Zeitdauer, die ein Prozess im System verbringt, entspricht $T_r = T_c - T_a$
- **Normalisierte Turnaround Time** – Turnaround Time in Relation zur Service Time, wird berechnet als T_r / T_s

- Gegeben sei folgende Prozessmenge:

– Prozesse	A	B	C
– Arrival Time	0	1	2
– Service Time	3	2	1

- In welcher Reihenfolge werden die Prozesse bei folgenden Schedulingverfahren ausgeführt?
 - FCFS
 - SPN
 - RR
 - MLFB (UNIX)
- Wie lautet die Completion Time, Turnaround Time und normalisierte Turnaround Time?

FCFS

Prozess	A	B	C
Arrival	0	1	2
Service	3	2	1

Completion Time $T_c =$

Turnaround Time $T_r = T_c - T_a =$

normalisierte Turnaround Time $T_r / T_s =$

SPN

Prozess	A	B	C
Arrival	0	1	2
Service	3	2	1

Completion Time $T_c =$

Turnaround Time $T_r = T_c - T_a =$

normalisierte Turnaround Time $T_r / T_s =$

RR (Länge Timeslice = 1)

Prozess	A	B	C
Arrival	0	1	2
Service	3	2	1

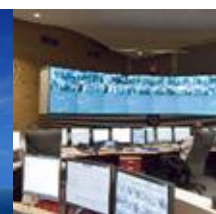
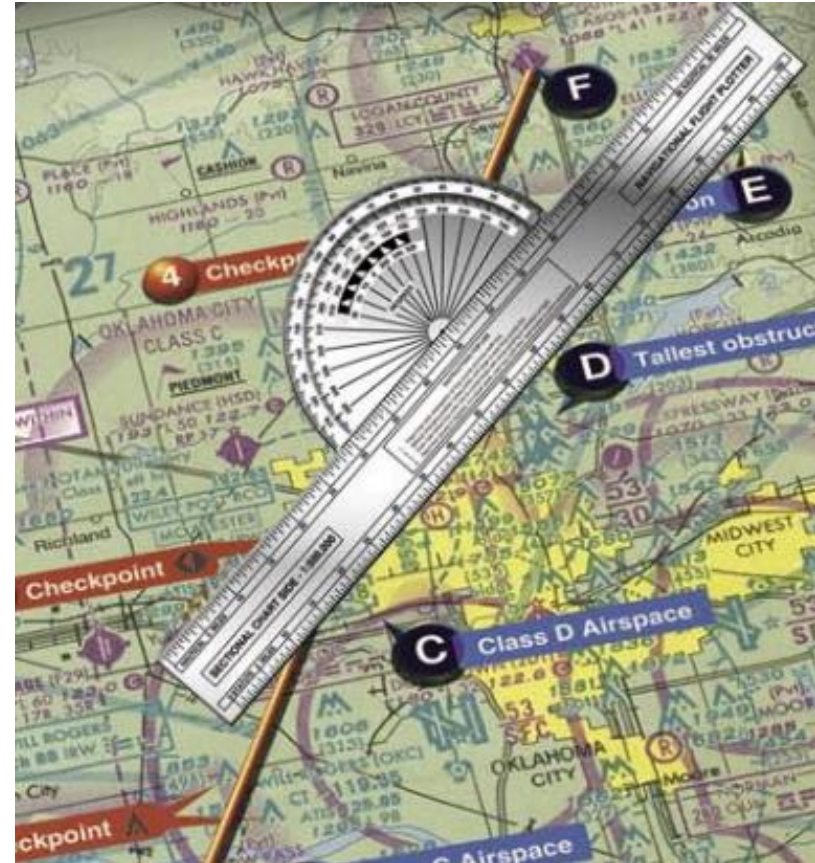
Completion Time $T_c =$

Turnaround Time $T_r = T_c - T_a =$

normalisierte Turnaround Time $T_r / T_s =$

Prozesse

- Prozesse
- Prozessorzuteilung (Scheduling)
 - Kooperativ vs. präemptiv
 - Ziele der Zuteilung
 - Strategien
 - First Come First Serve
 - Shortest Process Next
 - Round Robin
 - Statische Prioritäten
 - Multilevel Feedback
 - Metriken und Vergleich



Windows: Task Manager, Process Explorer und PsList

- Windows Taskmanager mit Strg-Shift-ESC aufrufen
- Karteikarte Prozesse wählen
- Sie erhalten eine Liste der im System aktiven Prozesse
- Eine Zeile mit der rechten Maustaste anklicken, Priorität festlegen wählen
- Die Prio-Stufe wird angezeigt und kann verändert werden
- Sehen Sie sich auch einmal den „Process Explorer“ an:
<http://technet.microsoft.com/de-de/sysinternals/bb795533>
- Rechtsklick-Properties auf einen Prozess zeigt auch Threads

Steuerung des der Prozessorzuteilung

Prozesspriorität in Unix vermindern

```
nice [ -n ] [ command ]
```

Das nice-Kommando startet einen Prozess mit verminderter Priorität.

```
renice n PID
```

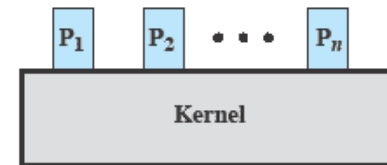
Das renice-Kommando vermindert die Priorität eines bereits laufenden Prozesses. Es ist nur eine Reduzierung der Prozess-Priorität möglich. Je höher der Wert n, desto geringer ist die Priorität.

Ausgabe von ps -lf

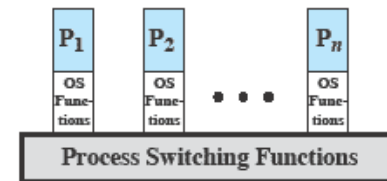
F	Systeminterne Flags
S	Status
UID	UID des Prozesseigentümers
PID	Prozess-ID
PPID	PID des Eltern-Prozesses (Parent PID)
C	Systeminterner Scheduling-Parameter
PRI	Aktuelle Priorität des Prozesses (je kleiner desto höher die Priorität)
NI	Nice-Parameter für die Prioritätsberechnung
ADDR	Hauptspeicher- oder Plattenadresse des Prozesses
SZ	Größe des Prozesses im Hauptspeicher in kByte
WCHAN	Adresse eines Events, auf den der Prozess wartet
STIME	Uhrzeit, wann der Prozess gestartet wurde (Startzeit des Prozesses)
TTY	Zugeordnetes Pseudo-Terminal
TIME	Bisher verbrauchte CPU-Zeit
CMD	Name des Kommandos, das den Prozess gestartet hat

Ist das Betriebssystem auch ein Prozess?

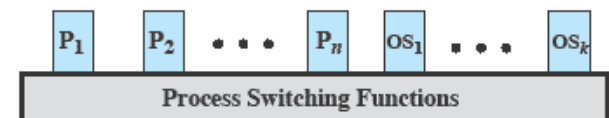
- a) Nein, Prozesse existieren nur für Anwendungsprogramme. Das Betriebssystem wird im Kernelmodus des Prozessors ausgeführt.
- b) Jein, Betriebssystemfunktionen werden im Kontext von User-Prozessen ausgeführt → kein Prozesswechsel notwendig (→UNIX)
- c) Ja, mit Ausnahme der Prozessumschaltung wird das Betriebssystem als Menge von Prozessen (mit speziellen) Privilegien realisiert. → siehe auch VL11, Microkernel



(a) Separate kernel



(b) OS functions execute within user processes



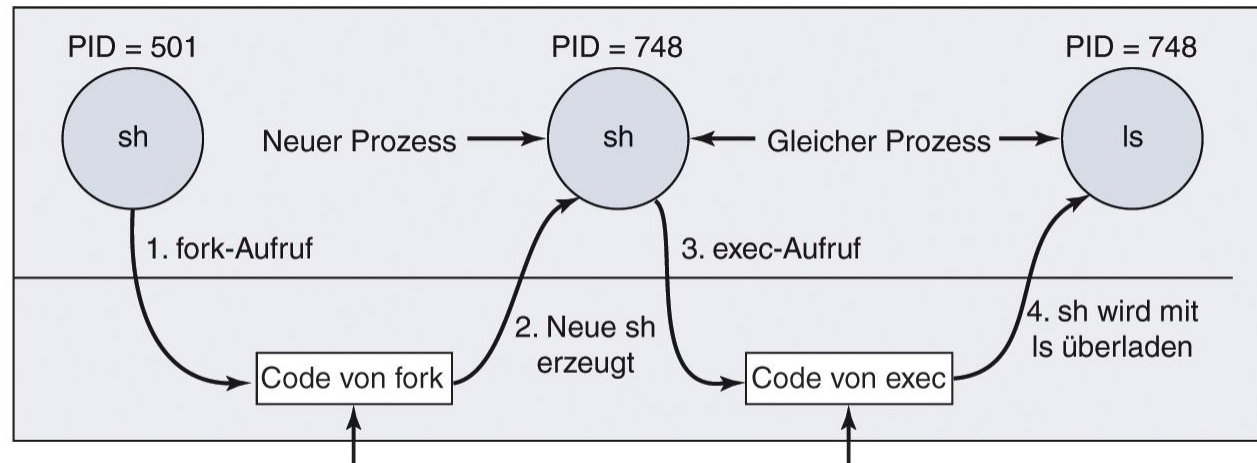
(c) OS functions execute as separate processes

Prozesserzeugung – fork, exec, wait

- Die Shell startet mit jedem Kommando einen neuen Prozess, was in drei Schritten abläuft
 1. *fork* - Ein neuer (Sohn-)Prozess wird zunächst als eine genaue Kopie (Klon) des ursprünglichen (Vater-)Prozesses mit neuer PID erzeugt
 2. *exec* - Der Sohn-Prozess wird überlagert durch den Code und die Daten des auszuführenden Befehls.
 3. *wait* - Der Vater-Prozess (d.h. Shell) wartet auf die Beendigung des Sohn-Prozesses (gestartetes Kommando).
- Der Aufruf *exec* kann auch direkt über die Shell erfolgen
exec Befehl

Dann wird die Shell durch den Befehl *ersetzt* (da kein *fork* stattfindet). *exec* kehrt bei Erfolg nicht zurück. Terminiert der gestartete Befehl, dann schließt sich auch die Terminalemulation.

- fork* und *exec* am Beispiel `ls`



Task-Struktur für Kindprozess belegen
Task-Struktur des Kindprozesses mit
der des Elternprozesses füllen
Stack und Benutzerbereich von
Kindprozess erzeugen
Benutzerbereich von Elternprozess
kopieren
PID für Kindprozess reservieren
Kind- und Elternprozess teilen sich
Textsegment
Seitentabellen für Stack und Daten
kopieren
Geöffnete Dateien werden gemeinsam
benutzt
Register des Elternprozesses zu
Kind kopieren

Ausführbares Programm finden
Ausführungsrechte prüfen
Header lesen und prüfen
Argumente und Umgebung in Kern
kopieren
Alten Adressraum freigeben
Neuen Adressraum belegen
Argumente und Umgebung auf Stack
kopieren
Signale zurücksetzen
Register initialisieren

```
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#define MAXLINE 4096 /* max line length */

int main(void)
{
    char    buf[MAXLINE];
    pid_t   pid;
    int     status;
    printf("%% "); /* print prompt (printf requires %% to print %) */
    while (fgets(buf, MAXLINE, stdin) != NULL) {
        buf[strlen(buf) - 1] = 0; /* replace newline with null */
        if ((pid = fork()) < 0) {
            perror("fork error");
            exit(1);
        }
        else if (pid == 0) { /* child */
            execlp(buf, buf, (char *) 0);
            printf("couldn't execute: %s\n", buf);
            exit(127);
        }
    }
}
```

```
/* parent */
    if ((pid = waitpid(pid, &status, 0)) < 0) {
        perror("waitpid error");
        exit(1);
    }

    printf("%% ");
}
exit(0);
}

/* Aus Richard Stevens: Advanced Programming in the Unix Environment */
```

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <sys/types.h>
int main(void)
{
    pid_t    pid;
    int      status;
    printf("Programm gestartet\n");
    pid = fork();          /* Kindprozess erzeugen */
    if (pid < 0) {          /* Fehler in fork aufgetreten */
        perror("fork error"); /* Fehlermeldung ausgeben */
        exit(1);           /* Programm mit Exitstatus 1 beenden */
    }
    else if (pid == 0) {    /* Kindprozess */
        sleep(1);           /* 1 sec schlafen */
        printf("Kindprozess mit PID %d\n", getpid());
        exit(100);          /* Kindprozess mit Exitstatus 100 terminieren */
    }
}
```

Anhang: Unix-Prozesse (2)

```
/* Elternprozess */
sleep(2);                /* 2 sec schlafen */
printf("Elternprozess mit PID %d\n", getpid());
printf("Kindprozess hat PID %d \n",pid);

pid = wait(&status);      /* Auf die Terminierung des Kindprozesses warten */
if (pid > 0) {            /* wait war erfolgreich */
    printf("Kindprozess PID %d ",pid);

    if (WIFEXITED(status)) /* Kind hat sich durch exit() selbst beendet */
        printf("mit Exit-Status %d ", status >> 8);

    printf("terminiert\n");
}
return(0);
}
```